



Ruben Apablaza Muñoz

PROYECTO
LABS
CIBERSEGURIDAD

LAB10
Automatización de análisis
SOC con Python

– 0Zone –

⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecute scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (`sudo`) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.

Contenido

Introducción	4
Requisito de operación	5
PROYECTO 10: Automatizar análisis de SOC con Python.	7
1. Ubicar logs que utilizaremos	8
2. Crear un archivo de práctica	9
3. Script 1: Contar intentos SSH fallidos	10
4. Script 2: Contar IP's que fallan más de X veces	12
5. Script 3: Detectar accesos fuera de horario	14
5.1 Generar un "mini-reporte automático"	15
6. Script 4: Conexiones a puertos específicos.....	17
7. Script 5: Procesos sospechosos	18
8. Script 6: Archivos que cambian sin razón	20
8.1 Guardar esos hashes como "baseline"	20
8.2 En el futuro, volvemos a ejecutar:.....	20
8.3 Comparar baseline vs nuevos:	21
8.4 Para automatizar la comparación	21
9. Script 7: Buscar ejecuciones sudo sospechosas.....	22
10. Script 8: Intentos de cambiar permisos o archivos del sistema	24
11. Script 9. Recolección de direcciones IP que acceden al servidor	29
12. Script 10: Detectar patrones de escaneo (nmap, masscan, etc.)	30
12.1 Activar logs de paquetes en tiempo real	32
12.1.1 Activar logging en IPTABLES.....	32
12.1.2 Hacer persistente la regla	33
12.1.3 Volver a correr el script	33
13. Script 11: Monitoreo de uso de red en tiempo real	34

14. Script 12: Monitoreo de usuarios.....	36
15. Script 13: Revisar actividad del firewall (UFW).....	39
15.1 Instalación de UFW	39
15.1.1 Revisar si UFW está instalado	39
15.1.2 Verificar si el paquete existe	39
15.1.3 Ver si el archivo de log existe o si está en otro lado	39
15.1.4 Activar logging de UFW (si esta apagado)	40
15.1.5 Si UFW no está instalado.....	40
15.1.6 Verificar si UFW está realmente activo en Ubuntu	40
15.1.7 Comprobar si apareció el log	41
15.2 Creación del script para revisar UFW	42
16. Script 14: Monitorear archivos de crontab por cambios.....	45
17. Script 15: Buscar palabras clave en logs (IOCs básicos)	53
18. Script 16: SOC completo.....	55
19. Conclusiones	88

Introducción

En el panorama actual de la ciberseguridad, los Centros de Operaciones de Seguridad (SOC) se enfrentan al desafío constante de mitigar amenazas en entornos corporativos cada vez más complejos e interconectados. Para un analista de seguridad, la capacidad de procesar, auditar e interpretar los registros de eventos (*logs*) del sistema es una competencia crítica; sin embargo, la revisión manual de miles de líneas de datos resulta ineficiente y propensa al error humano, generando fenómenos críticos como la “fatiga de alertas”.

Para solucionar esto, el mercado ofrece plataformas SIEM y XDR consolidadas (como **Wazuh**, Splunk o Elastic Security) que automatizan gran parte de este trabajo mediante reglas preconfiguradas. No obstante, depender exclusivamente de soluciones comerciales o de “caja negra” puede limitar la capacidad de adaptación de un analista ante escenarios imprevistos o firmas de ataque personalizadas. Por ello, la verdadera diferencia entre un **operador de herramientas** y un **especialista en ciberseguridad** radica en comprender a fondo la lógica subyacente de la detección.

Este proyecto nace con ese propósito fundamental.

A lo largo de este documento, se detalla la construcción paso a paso de un ecosistema de scripts en Python diseñados para auditar registros reales en un entorno de servidor Linux (Ubuntu Server). Al programar nuestra propia lógica para identificar ataques de fuerza bruta por SSH, accesos fuera de horario, alteraciones de archivos (FIM) o escaneos de red, el lector no solo aprende a detectar Indicadores de Compromiso (IOCs), sino que asimila el comportamiento real de los flujos de datos. Esta aproximación práctica dota al profesional de la autonomía necesaria para entender qué ocurre “bajo el capó” de herramientas como Wazuh, permitiéndole en el futuro optimizar reglas, crear filtros de alta fidelidad y diseñar soluciones de seguridad a medida.

Requisito de operación

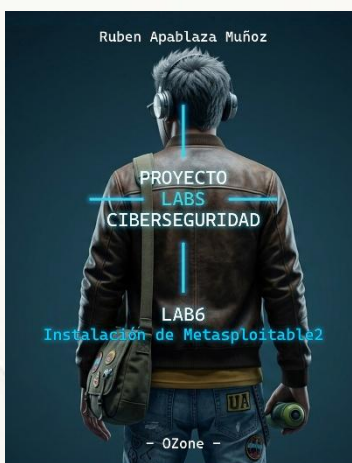
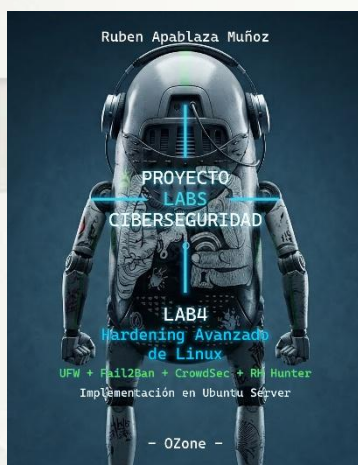
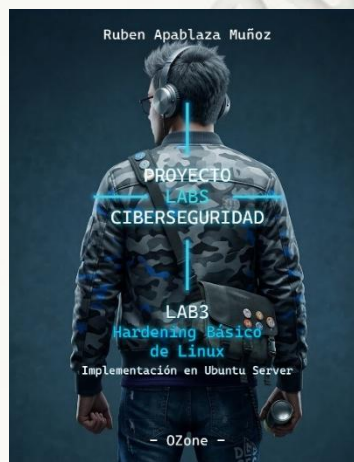
Esta guía es la continuación estratégica de los laboratorios:

- **Lab 1: Instalación y preparación de Ubuntu Server.**
- **Lab 2: Análisis táctico de logs en sistemas Linux.**
- **Lab 3: Hardening Básico de Linux.**
- **Lab 4: Hardening AVANZADO de Linux: UFW + Fail2Ban + CrowdSec + RK Hunter.**
- **Lab 5: Instalación de Kali Linux**
- **Lab 6: Instalación de Metasploitable2**
- **Lab 7: Escaneo de red con Nmap**
- **Lab 8: Sniffing de red con Wireshark**
- **Lab 9: Cracking de hashes**

En ciberseguridad, la integridad del entorno es fundamental. Para garantizar que las acciones que vamos a realizar hoy sean consistentes y que las configuraciones de red no interfieran con tus hallazgos, es **altamente recomendable** haber completado la fase previa.

Si aún no tienes tu entorno listo o quieres asegurarte de que tu máquina cumple con los estándares de este proyecto, puedes realizar ambos laboratorios aquí:

Haz *click* en la imagen para acceder al **Lab** correspondiente.



PROYECTO 10: Automatizar análisis de SOC con Python.

Crear scripts simples en Python que lean logs reales de tu **Ubuntu Server** y encuentren patrones como:

- Intentos de login fallidos
- Conexiones ssh sospechosas
- Accesos repetidos
- Orígenes raros
- Actividad fuera de horario

Esto es lo que hace un SOC junior: revisar logs, automatizar tareas repetitivas y generar alertas simples con herramientas pero aquí aprenderemos a crear herramientas con Python para entender su funcionamiento y alcance.

Nuestra VM Ubuntu ya está en modo bridge, así que estamos listo.

1. Ubicar logs que utilizaremos

En Ubuntu:

- Auth y SSH
/var/log/auth.log
- Sistema general
/var/log/syslog

Para este proyecto usaremos **auth.log**, que ya existe sin instalar nada, podemos darle una “ojeada” rápida con:

```
sudo tail -n 50 /var/log/auth.log
```

```
ozone@ubuntu:~$ sudo tail -n 50 /var/log/auth.log
[sudo] password for ozone:
2026-02-23T16:35:01.773019-03:00 ubuntu: CRON[1867]: pam_unix(cron:session): session closed for user root
2026-02-23T16:45:01.796471-03:00 ubuntu: CRON[1876]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-02-23T16:45:01.810573-03:00 ubuntu: CRON[1876]: pam_unix(cron:session): session closed for user root
2026-02-23T16:55:01.832794-03:00 ubuntu: CRON[2006]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-02-23T16:55:01.847999-03:00 ubuntu: CRON[2006]: pam_unix(cron:session): session closed for user root
2026-02-23T17:05:01.871795-03:00 ubuntu: CRON[2025]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-02-23T17:05:01.883655-03:00 ubuntu: CRON[2025]: pam_unix(cron:session): session closed for user root
2026-02-23T17:15:01.906953-03:00 ubuntu: CRON[2049]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-02-23T17:15:01.921329-03:00 ubuntu: CRON[2049]: pam_unix(cron:session): session closed for user root
2026-02-23T17:17:01.941534-03:00 ubuntu: CRON[2054]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-02-23T17:17:01.955064-03:00 ubuntu: CRON[2054]: pam_unix(cron:session): session closed for user root
2026-02-23T17:24:55.000760-03:00 ubuntu: sshd[1614]: Received disconnect from 192.168.0.33 port 47150:11: disconnected by user
2026-02-23T17:24:55.000760-03:00 ubuntu: sshd[1614]: Disconnected from user ozone, 192.168.0.33 port 47150
```

2. Crear un archivo de práctica

Dentro de nuestro Ubuntu Server (los colores de la imagen son porque yo lo hago conectado por ssh desde VS Code) creamos un directorio llamado **“soc10”** en el que copiaremos y pegaremos el archivo de logs (auth.log):

```
mkdir ~/soc10  
cd ~/soc10  
cp /var/log/auth.log auth.log
```

Así podemos practicar sin tocar el log real.



```
ozone@ubuntuuserver:~$ mkdir ~/soc10  
ozone@ubuntuuserver:~$ ls  
hash_prueba.txt  lab  soc10  
ozone@ubuntuuserver:~$ cd soc10  
ozone@ubuntuuserver:~/soc10$ cp /var/log/auth.log auth.log  
ozone@ubuntuuserver:~/soc10$ ls  
auth.log  
ozone@ubuntuuserver:~/soc10$
```

3. Script 1: Contar intentos SSH fallidos

Monitorear los intentos fallidos de SSH es crucial porque es nuestro **sistema de alerta temprana** para detectar ataques de fuerza bruta y escaneos automatizados.

Básicamente, permite identificar y bloquear a un atacante **antes** de que adivine la contraseña correcta y tome el control total del servidor. Si no contamos esos intentos, estamos operando a ciegas contra las amenazas.

A continuación vamos a crear nuestro script en Python que contará intentos fallidos de inicio de sesión por ssh. Si no tienes intentos fallidos registrados en el log, no devolverá nada por lo que quizás deberías seguir mi ejemplo y conectarte por ssh y generar algunas contraseñas incorrectas para que queden en el registro y entonces copiar el archivo auth.log del paso anterior con estos intentos fallidos registrados.

Si ya lo tienes, continuamos con nuestro script de Python.

A mí personalmente se me hace más sencillo escribir el script en VS Code, tanto por los colores y porque no decirlo, por el autocompletado que puede ayudar a ahorrar tiempo, siempre y cuando entendamos lo que estamos haciendo con ayuda de la IA y no vamos por ahí dejando brechas de seguridad abiertas en nuestro script.

Creemos el script que es básicamente un bucle for que buscara "Failed password" en cada línea del "auth.log" como se ve a continuación.

```
import re

with open("auth.log", "r") as f:
    for linea in f:
        if "Failed password" in linea:
            print(linea.strip())
```

A continuación crearemos un archivo de Python en nuestra maquina Ubuntu con

```
nano ssh_fallidos.py
```



```
GNU nano 7.2
import re

with open("auth.log", "r") as f:
    for linea in f:
        if "Failed password" in linea:
            print(linea.strip())
```

y simplemente pegamos las líneas en su interior como muestra la imagen anterior, guardamos y cerramos. Cabe señalar en este punto, que puedo copiar y pegar porque estoy en un homelab con máquinas virtuales como hemos visto en los laboratorios anteriores, si estoy en un entorno distinto recuerden que podemos utilizar métodos para compartir archivos como los vistos en otros documentos de los que hemos revisados como crear un servidor, compartir por scp y otras formas.

A continuación ejecutamos nuestro script con `y` y podemos ver que nos entrega resultados de intentos fallidos encontrados en el `auth.log`

```
python3 ssh_fallidos.py
```

```
ozone@ubuntuserver:~/soc10$ python3 ssh_fallidos.py
2026-02-26T15:40:29.764750-03:00 ubuntuserver sshd[1723]: Failed password for ozone from 192.168.0.33 port 49718 ssh2
2026-02-26T15:40:35.758063-03:00 ubuntuserver sshd[1723]: Failed password for ozone from 192.168.0.33 port 49718 ssh2
```

Recuerda: Si no hay intentos fallidos no retornará nada... en este caso yo me conecte por ssh también desde VS Code a la máquina virtual Ubuntu e ingrese un par de veces contraseñas incorrectas que se pueden apreciar en el registro.

4. Script 2: Contar IP's que fallan más de X veces

Tener el conteo por IP es vital porque permite automatizar la defensa inmediata (como usar Fail2Ban).

Al identificar la IP específica que está atacando, el sistema puede bloquearla por completo en el firewall, cortando el ataque de raíz antes de que comprometa el servidor y diferenciando a un atacante real de un usuario legítimo que solo olvidó su contraseña.

Creemos nuestro script en Python:

```
import re
from collections import Counter

patron = re.compile(r"Failed password.*from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)")

ips = []

with open("auth.log") as f:
    for linea in f:
        m = patron.search(linea)
        if m:
            ips.append(m.group(1))

conteo = Counter(ips)

for ip, cantidad in conteo.items():
    if cantidad >= 3:
        print(f"{ip} con {cantidad} intentos fallidos")
```

**** Para que el ejercicio funcione, previamente se debe intentar ingresar por ssh desde distintas direcciones ip con credenciales erróneas para que el archivo "auth.log" tenga el registro de estos intentos fallidos, si no, es probable que nuestro script no devuelva nada.*

Por este motivo tengo que volver a copiar el archivo auth.log a nuestro directorio de pruebas "soc10" para que registre estos cambios.

```

ozone@ubuntuuserver:~$ ls
auth.log  hash_prueba.txt  lab  soc10
ozone@ubuntuuserver:~$ cd soc19
-bash: cd: soc19: No such file or directory
ozone@ubuntuuserver:~$ cd soc10
ozone@ubuntuuserver:~/soc10$ ls
auth.log  ssh_fallidos.py
ozone@ubuntuuserver:~/soc10$ rm auth.log
ozone@ubuntuuserver:~/soc10$ ls
ssh_fallidos.py
ozone@ubuntuuserver:~/soc10$ cp /var/log/auth.log auth.log
ozone@ubuntuuserver:~/soc10$ ls
auth.log  ssh_fallidos.py
ozone@ubuntuuserver:~/soc10$

```

y al igual que en el caso anterior copiamos las líneas a un archivo Python creado en Ubuntu

`nano ip_repetidas.py`

```

GNU nano 7.2 ip_repetidas.py *
import re
from collections import Counter

patron = re.compile(r"Failed password.*from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)")

ips = []

with open("auth.log") as f:
    for linea in f:
        m = patron.search(linea)
        if m:
            ips.append(m.group(1))

conteo = Counter(ips)

for ip, cantidad in conteo.items():
    if cantidad >= 3:
        print(f"{ip} con {cantidad} intentos fallidos")

```

Guardamos y corremos nuestro script.

`python3 ip_repetidas.py`

```

ozone@ubuntuuserver:~/soc10$ python3 ip_repetidas.py
192.168.0.33 con 9 intentos fallidos
192.168.0.38 con 8 intentos fallidos
ozone@ubuntuuserver:~/soc10$

```


5. Script 3: Detectar accesos fuera de horario

Detectar accesos fuera de horario es fundamental porque rompe el patrón de comportamiento normal del usuario, lo que suele ser el indicador más claro de una cuenta comprometida (un atacante usando credenciales válidas) o de una amenaza interna (un empleado intentando extraer información sin supervisión).

En ciberseguridad, un inicio de sesión legítimo a las 3:00 a.m. de un domingo es, hasta que se demuestre lo contrario, un incidente de alto riesgo.

Creemos nuestro script con:

```
nano horario_raro.py
```

```
import datetime

for linea in open("auth.log"):
    if "Accepted password" not in linea and "session opened" not in linea:
        continue

    fecha_bruta = linea.split()[0]

    # Intento formato ISO
    try:
        fecha = datetime.datetime.fromisoformat(fecha_bruta)
    except:
        # Intento formato clasico
        try:
            partes = linea.split()
            fecha_txt = " ".join(partes[:3])
            fecha = datetime.datetime.strptime(fecha_txt, "%b %d %H:%M:%S")
        except:
            continue

    if fecha.hour < 6 or fecha.hour > 22:
        print("Acceso fuera de horario:", linea.strip())
```

Para poder demostrar que funciona, tuve que mentir en este caso ya que como es un servidor relativamente limpio y no está en producción lo cierto es que no hay accesos fuera de horario entre las 22:00 y las 06:00 hrs (*porque los niños buenos nos acostamos temprano XD*) y por lo tanto al correr el script no devuelve nada, entonces solo para mostrar que funciona hice un pequeño ajuste en esta línea para que detectara ingresos como si fuesen fuera de horario.

```
if fecha.hour > 6 or fecha.hour < 22:
```

y el resultado:

```
ozone@ubuntu:~/soc10$ python3 horario_raro.py
Acceso fuera de horario: 2026-02-23T16:05:00.645138-03:00 ubuntu:server CRON[1381]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Acceso fuera de horario: 2026-02-23T16:05:28.442463-03:00 ubuntu:server login[1120]: pam_unix(login:session): session opened for user ozone(uid=1000) by ozone(uid=0)
Acceso fuera de horario: 2026-02-23T16:05:28.529842-03:00 ubuntu:server (systemd): pam_unix(systemd-user:session): session opened for user ozone(uid=1000) by ozone(uid=0)
Acceso fuera de horario: 2026-02-23T16:06:44.369977-03:00 ubuntu:server sshd[1556]: Accepted password for ozone from 192.168.0.33 port 47150 ssh2
Acceso fuera de horario: 2026-02-23T16:06:44.377747-03:00 ubuntu:server sshd[1556]: pam_unix(sshd:session): session opened for user ozone(uid=1000) by ozone(uid=0)
Acceso fuera de horario: 2026-02-23T16:07:09.739429-03:00 ubuntu:server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ozone(uid=1000)
Acceso fuera de horario: 2026-02-23T16:12:49.947044-03:00 ubuntu:server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ozone(uid=1000)
Acceso fuera de horario: 2026-02-23T16:12:49.962466-03:00 ubuntu:server su[1678]: pam_unix(su-l:session): session opened for user prueba(uid=1001) by ozone(uid=0)
```

Después de correr el script lo devolvemos a su forma original.

5.1 Generar un “mini-reporte automático”

Si queremos generar un mini-reporte automático

```
nano informe.py
```

```
import subprocess

print("=== Intentos SSH fallidos ===")
subprocess.run(["python3", "ssh_fallidos.py"])

print("\n=== IPs con fuerza bruta ===")
subprocess.run(["python3", "ip_repetidas.py"])

print("\n=== Accesos fuera de horario ===")
subprocess.run(["python3", "horario_raro.py"])
```

y lo ejecutamos con:

`python3 informe.py`

```
ozone@ubuntu:~/soc10$ python3 informe.py
=== Intentos SSH fallidos ===
2026-02-26T15:40:29.764750-03:00 ubuntu: sshd[1723]: Failed password for ozone from 192.168.0.33 port 49718 ssh2
2026-02-26T15:40:35.758063-03:00 ubuntu: sshd[1723]: Failed password for ozone from 192.168.0.33 port 49718 ssh2
2026-02-26T17:56:19.784115-03:00 ubuntu: sshd[1461]: Failed password for ozone from 192.168.0.38 port 41806 ssh2
2026-02-26T17:56:26.964799-03:00 ubuntu: sshd[1461]: Failed password for ozone from 192.168.0.38 port 41806 ssh2
2026-02-26T17:56:40.981435-03:00 ubuntu: sshd[1463]: Failed password for ozone from 192.168.0.38 port 59714 ssh2
2026-02-26T17:56:45.696721-03:00 ubuntu: sshd[1463]: Failed password for ozone from 192.168.0.38 port 59714 ssh2
2026-02-26T17:56:57.398527-03:00 ubuntu: sshd[1465]: Failed password for ozone from 192.168.0.38 port 45212 ssh2
2026-02-26T17:57:03.892769-03:00 ubuntu: sshd[1465]: Failed password for ozone from 192.168.0.38 port 45212 ssh2
2026-02-26T17:57:19.694169-03:00 ubuntu: sshd[1471]: Failed password for ozone from 192.168.0.38 port 60400 ssh2
2026-02-26T17:57:25.537358-03:00 ubuntu: sshd[1471]: Failed password for ozone from 192.168.0.38 port 60400 ssh2
2026-02-26T17:58:39.906744-03:00 ubuntu: sshd[1544]: Failed password for ozone from 192.168.0.33 port 39820 ssh2
2026-02-26T17:58:44.387692-03:00 ubuntu: sshd[1544]: Failed password for ozone from 192.168.0.33 port 39820 ssh2
2026-02-26T17:58:51.826572-03:00 ubuntu: sshd[1546]: Failed password for ozone from 192.168.0.33 port 39821 ssh2
2026-02-26T17:58:56.821372-03:00 ubuntu: sshd[1546]: Failed password for ozone from 192.168.0.33 port 39821 ssh2
2026-02-26T17:59:05.468763-03:00 ubuntu: sshd[1548]: Failed password for ozone from 192.168.0.33 port 39826 ssh2
2026-02-26T17:59:11.842489-03:00 ubuntu: sshd[1548]: Failed password for ozone from 192.168.0.33 port 39826 ssh2
2026-02-26T17:59:28.843849-03:00 ubuntu: sshd[1550]: Failed password for ozone from 192.168.0.33 port 39827 ssh2

=== IPs con fuerza bruta ===
192.168.0.33 con 9 intentos fallidos
192.168.0.38 con 8 intentos fallidos

=== Accesos fuera de horario ===
ozone@ubuntu:~/soc10$
```

Si a partir de lo anterior queremos generar un archivo con extensión `.txt`

`python3 informe.py > reporte.txt`

Si queremos programar ejecución automática con cron

Primero abriremos cron:

`crontab -e`

y agregamos lo siguiente:

```
*/10 * * * * cd /home/TU_USUARIO/soc10 && cp /var/log/auth.log
auth.log && python3 informe.py > reporte.txt
```

Esto ejecuta cada 10 minutos un mini “SOC automatico”.

En este punto se debe dejar en claro que Python en ciberseguridad se utiliza para mucho mas que leer intentos de inicio de sesión por ssh.

A continuación dejo algunos ejemplos de **COSAS IMPORTANTES** que deberíamos monitorear con Python.

6. Script 4: Conexiones a puertos específicos

Ejemplo: Detectar si alguien se conecta al puerto 22, 80, 443 u otro que **no debería** tener tráfico.

Comando base en Linux:

```
ss -tulpn
```

Con un script como el siguiente en Python:

```
import subprocess

resultado = subprocess.check_output(["ss", "-tulpn"]).decode()

for linea in resultado.splitlines():
    if ":22" in linea:
        print("Conexion en puerto 22:", linea)
```

y el resultado:

```
ozone@ubuntuuserver:~/soc10$ python3 conexion_puertos_especificos.py
Conexion en puerto 22: tcp    LISTEN 0      4096      0.0.0.0:22      0.0.0.0:*
Conexion en puerto 22: tcp    LISTEN 0      4096      [::]:22      [::]:*
ozone@ubuntuuserver:~/soc10$
```

Esto sirve para detectar:

- Puertos abiertos sin razón.
- Servicios levantados sin permiso.
- Procesos raros escuchando.

7. Script 5: Procesos sospechosos

Podemos generar un script como “Monitor de procesos” para detectar procesos nuevos o raros.

Instalamos **psutil** que no viene por defecto en Ubuntu con `y` creamos el script:

```
sudo apt update
```

```
sudo apt install python3-psutil -y
```

```
import psutil
import os

mi_pid = os.getpid() # para no marcar el propio script

# procesos que vale la pena vigilar
sospechosos = ["nc", "ncat", "netcat", "bash -i", "reverse", "perl", "php",
"python3", "meterpreter"]

for p in psutil.process_iter(["pid", "name", "cmdline"]):
    try:
        pid = p.info["pid"]
        nombre = p.info["name"] or ""
        cmd = " ".join(p.info["cmdline"]) if p.info["cmdline"] else ""

        if pid == mi_pid:
            continue # ignorar el propio proceso

        # reglas simples
        if any(s in nombre.lower() for s in sospechosos):
            print(f"[ALERTA] Proceso sospechoso: PID={pid}, NAME={nombre},
CMD={cmd}")

        if any(s in cmd.lower() for s in sospechosos):
            print(f"[ALERTA] Proceso sospechoso (cmdline): PID={pid},
NAME={nombre}, CMD={cmd}")

    except Exception:
        pass
```

ejecutamos y acá tenemos la salida en mi caso:

```
ozone@ubuntu-server:~/soc10$ python3 procesos_sospechosos.py
[ALERTA] Proceso sospechoso: PID=646, NAME=systemd-timesyncd, CMD=/usr/lib/systemd/systemd-timesyncd
[ALERTA] Proceso sospechoso (cmdline): PID=646, NAME=systemd-timesyncd, CMD=/usr/lib/systemd/systemd-timesyncd
[ALERTA] Proceso sospechoso (cmdline): PID=857, NAME=unattended-upgr, CMD=/usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
ozone@ubuntu-server:~/soc10$
```

Esto sirve para:

- Detectar backdoors
- Shells raras
- Procesos que no deberían correr en un servidor



8. Script 6: Archivos que cambian sin razón

Podemos utilizar un script para comprobar la integridad básica, muy útil para ver cambios en archivos críticos.

```
import hashlib

def hash_archivo(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        h.update(f.read())
    return h.hexdigest()

h1 = hash_archivo("/etc/passwd")
h2 = hash_archivo("/etc/shadow")

print(h1, h2)
```

Es importante señalar que estos archivos están “sanos” ya que nuestro sistema está relativamente nuevo.

```
ozone@ubuntuuserver:~/soc10$ sudo python3 cambios_archivos_criticos.py
[sudo] password for ozone:
20cccd34f17ec3c65b78fd668e94b4          f50aab 32c6052376628d          27f
7605c71          7843
ozone@ubuntuuserver:~/soc10$
```

8.1 Guardar esos hashes como “baseline”

Crea un archivo donde guardes los valores actuales (estado sano de tu sistema).

```
sudo python3 cambios_archivos_criticos.py >
baseline_hashes.txt
```

Listo. Ese archivo es nuestra referencia para comparar.

8.2 En el futuro, volvemos a ejecutar:

```
sudo python3 cambios_archivos_criticos.py > hashes_nuevos.txt
```

8.3 Comparar baseline vs nuevos:

```
diff baseline_hashes.txt hashes_nuevos.txt
```

- Si no hay diferencia:
"Sistema integro. Todo bien."
- Si hay diferencia:
"Algún archivo critico fue modificado."

Podemos guardar estos hashes y compararlos cada cierto tiempo.

8.4 Para automatizar la comparación

Podemos crear un script que compare automáticamente:

```
nano verificar_integridad.py
```

Con el siguiente script:

```
import subprocess

# generar hash actual
actual = subprocess.check_output(["python3",
    "cambios_archivos_criticos.py"]).decode()

# leer baseline creado en el paso 8.1
with open("baseline_hashes.txt") as f:
    baseline = f.read()

if actual.strip() != baseline.strip():
    print("ALERTA: algun archivo critico cambio")
else:
    print("Sin cambios, sistema integro")
```

Ejecutamos:

```
sudo python3 verificar_integridad.py
```

En este caso como no se han realizado cambios que levanten alertas por lo tanto la salida es normal.

```
ozone@ubuntu1704:~/soc10$ nano verificar_integridad.py
ozone@ubuntu1704:~/soc10$ sudo python3 verificar_integridad.py
Sin cambios, sistema integro
```


9. Script 7: Buscar ejecuciones sudo sospechosas

Muy común para detectar escalamiento.

nano sudo_sospechoso.py

```
import datetime
import re

usuarios_sospechosos = ["www-data", "nobody", "daemon"]
comandos_peligrosos = ["bash", "sh", "python", "perl", "nano", "vi",
"chmod", "chown", "useradd", "usermod"]

with open("/var/log/auth.log") as f:
    for linea in f:
        if "sudo" not in linea:
            continue

        # extraer timestamp ISO 8601
        # ejemplo: 2026-02-23T16:07:09.733413-03:00
        match = re.match(r"^(\d{4}-\d{2}-\d{2}T[\d:\.\-+]+)", linea)
        if match:
            timestamp = match.group(1)
            try:
                hora = datetime.datetime.fromisoformat(timestamp)
            except ValueError:
                continue
        else:
            continue

        # extraer usuario
        usuario = None
        partes = linea.split()
        for p in partes:
            if "USER=" in p:
                usuario = p.split("=")[1]

        # extraer comando sudo
        if "COMMAND=" in linea:
            cmd = linea.split("COMMAND=")[1].strip()
        else:
            cmd = ""

        alerta = False
        razones = []
```

```

# 1. usuario que no deberia usar sudo
if usuario in usuarios_sospechosos:
    alerta = True
    razones.append("usuario no deberia usar sudo")

# 2. comandos peligrosos
if any(c in cmd for c in comandos_peligrosos):
    alerta = True
    razones.append("sudo ejecuto comando delicado")

# 3. uso nocturno (00:00-06:00)
if hora.hour < 6:
    alerta = True
    razones.append("sudo fuera de horario")

# mostrar alertas reales solamente
if alerta:
    print("[ALERTA]", ", ".join(razones))
    print(" ", linea.strip(), "\n")

```

y cuando lo ejecutamos nos muestra

`sudo python3 sudo_sospechosos.py`

```

ozone@ubuntu:~$ nano sudo_sospechoso.py
ozone@ubuntu:~$ sudo python3 sudo_sospechoso.py
[ALERTA] sudo ejecuto comando delicado
2026-02-23T16:18:38.326271-03:00 ubuntu: sudo:      ozone : TTY=pts/1 ; PWD=/home/ozone ; USER=root
; COMMAND=/usr/bin/cat /etc/shadow

[ALERTA] sudo ejecuto comando delicado
2026-02-23T16:22:43.604754-03:00 ubuntu: sudo:      ozone : TTY=pts/1 ; PWD=/home/ozone ; USER=root
; COMMAND=/usr/bin/cat /etc/shadow

[ALERTA] sudo ejecuto comando delicado
2026-02-26T20:27:37.213868-03:00 ubuntu: sudo:      ozone : TTY=pts/0 ; PWD=/home/ozone/soc10 ; USE
R=root ; COMMAND=/usr/bin/apt install python3-psutil -y

[ALERTA] sudo ejecuto comando delicado
2026-02-26T20:49:48.348068-03:00 ubuntu: sudo:      ozone : TTY=pts/0 ; PWD=/home/ozone/soc10 ; USE
R=root ; COMMAND=/usr/bin/python3 cambios_archivos_criticos.py

[ALERTA] sudo ejecuto comando delicado
2026-02-27T13:16:23.610774-03:00 ubuntu: sudo:      ozone : TTY=pts/0 ; PWD=/home/ozone/soc10 ; USE
R=root ; COMMAND=/usr/bin/python3 cambios_archivos_criticos.py

[ALERTA] sudo ejecuto comando delicado
2026-02-27T13:19:00.628790-03:00 ubuntu: sudo:      ozone : TTY=pts/0 ; PWD=/home/ozone/soc10 ; USE
R=root ; COMMAND=/usr/bin/python3 verificar_integridad.py

[ALERTA] sudo ejecuto comando delicado
2026-02-27T13:22:16.624484-03:00 ubuntu: sudo:      ozone : TTY=pts/0 ; PWD=/home/ozone/soc10 ; USE
R=root ; COMMAND=/usr/bin/python3 cambios_archivos_criticos.py

```

10. Script 8: Intentos de cambiar permisos o archivos del sistema

Detectar cambios en archivos del sistema o en sus permisos es crítico porque es la señal más clara de que un atacante ya logró entrar y está intentando consolidar su control. Específicamente, permite descubrir dos de las acciones más peligrosas en ciberseguridad:

- **Escalada de privilegios:** Modificar permisos para que un usuario común pueda actuar como administrador.
- **Instalación de Backdoors (puertas traseras):** Alterar archivos legítimos del sistema operativo para asegurar su acceso permanente, incluso si cambiamos las contraseñas.

Básicamente, si no vigilamos la integridad de los archivos, el atacante puede adueñarse del sistema de forma invisible y permanente.

Con el siguiente script podemos buscar actividad maliciosa en el historial de autenticación y seguridad de Linux:

```
archivos_criticos = ["/etc/passwd", "/etc/shadow", "/etc/sudoers"]

with open("/var/log/auth.log") as f:
    for linea in f:
        if "chmod" in linea or "chown" in linea:

            # sin sudo = mas sospechoso
            if "sudo" in linea:
                continue

            # usuario no root
            if "user=root" in linea:
                continue

            # solo archivos valiosos
            if not any(a in linea for a in archivos_criticos):
                continue

        print("[ALERTA REAL] Cambio sospechoso:", linea.strip())
```

Explicación para entenderlo:

- `archivos_criticos= [...]`:

Define la lista negra. Estos tres archivos controlan los usuarios (`passwd`), sus contraseñas cifradas (`shadow`) y quién tiene permisos de administrador (`sudoers`). Tocarlos sin autorización es tomar el control del servidor.

- `if "chmod" in linea or "chown" in linea::`

El script se activa únicamente si alguien intentó cambiar los permisos (`chmod`) o el dueño (`chown`) de algún archivo.

- `if "sudo" in linea: continue: (Filtro 1)`

Si el comando se ejecutó usando `sudo`, el script lo ignora y pasa a la siguiente línea. Asume que es una tarea administrativa legítima.

- `if "user=root" in linea: continue: (Filtro 2)`

Si la acción la hizo directamente el usuario `root` (el administrador supremo), también lo ignora por la misma razón.

- `if not any(a in linea for a in archivos_criticos): continue: (Filtro 3)`

Si los comandos `chmod` o `chown` se aplicaron a cualquier otro archivo que **no** sea uno de los tres archivos críticos (por ejemplo, un usuario cambiando los permisos de una foto en su carpeta personal), el script lo ignora.

- `print("[ALERTA REAL]..."):`

Si el evento sobrevivió a los tres filtros anteriores, significa que **un usuario común, sin usar sudo, intentó modificar un archivo crítico del sistema**. Eso es un comportamiento altamente sospechoso (un posible ataque) y por eso dispara la alerta.

El Impacto del “Filtro”: ¿Cómo limpia el ruido?

En un servidor real, se generan miles de líneas de logs cada hora. Si intentáramos revisar todo, sufriríamos de "fatiga de alertas" (ignorarías los avisos por cansancio). Este filtro funciona como un colador inteligente:

- **Instaladores y Operaciones administrativas → Ignorados:** Cuando instalamos un programa o el administrador del sistema hace mantenimiento, los comandos `chmod` o `chown` se ejecutan constantemente usando `sudo` o como `root`. El script los detecta, ve que son legítimos y los descarta inmediatamente.
- **Usuarios Normales + Archivos críticos → ALERTA:** Si un usuario común (que no debería tocar la configuración base) intenta alterar permisos en los archivos clave (`/etc/...`), el script detiene el colador, deja pasar ese evento y dispara la alarma.

El resultado: Eficiencia nivel SOC

Cuando logramos este equilibrio, nuestra herramienta “casera” adquiere capacidades de nivel empresarial:

- **Cero ruido y alertas basura:** No vamos a recibir un correo de alerta cada vez que el sistema se actualice o el administrador trabaje. El 99.9% de la actividad normal queda bloqueada en el filtro.
- **Detección de alta fidelidad:** Si el script emite una alerta, es porque algo anda muy mal. No hay falsos positivos; es una amenaza real que requiere atención inmediata.
- **Automatización total:** Al ser un script tan limpio y ligero, podemos programarlo con `cron` (el planificador de tareas de Linux) para que revise el archivo `auth.log` cada 5 minutos, convirtiendo nuestro servidor en un mini-SIEM automatizado.

Cabe señalar que herramientas como Wazuh traen esto incorporado de forma nativa y a nivel profesional.

En el mundo de los SIEM, esta función se divide en dos componentes que trabajan juntos:

1. **FIM (File Integrity Monitoring / Monitoreo de integridad de archivos):** Wazuh tiene un módulo específico que vigila archivos críticos (como /etc/passwd). Si detecta cualquier cambio de contenido, permisos (chmod) o dueño (chown), lo registra de inmediato.
2. **Motor de reglas (El “filtro”):** Wazuh cuenta con miles de reglas preconfiguradas basadas en políticas de seguridad globales. Estas reglas hacen exactamente lo mismo que nuestro script: filtran el ruido de los procesos legítimos del sistema y de los administradores, y disparan una alerta de "Alta Severidad" solo cuando el cambio es sospechoso o no autorizado.

Básicamente, nuestro script es la versión en código puro de lo que Wazuh hace a gran escala en miles de servidores simultáneamente.

Entonces ¿Por qué es importante entender el script?

Es importante por tres razones clave:

- **Entender los fundamentos:** Wazuh es una "caja negra" que hace magia; entender el script nos enseña la lógica real (qué archivos importan, qué comandos son peligrosos y cómo piensa un atacante).
- **Nos permite personalizar:** Si necesitamos vigilar un archivo interno de nuestra empresa que Wazuh no cubre por defecto, sabremos exactamente cómo crear nuestra propia regla lógica.
- **Diagnóstico de problemas:** Si un SIEM falla, deja de reportar o genera falsos positivos, el analista que entiende el código base es el único capaz de encontrar el error y arreglarlo.

En resumen: **Wazuh te hace eficiente, pero entender el script es lo que nos hace un verdadero profesional de la ciberseguridad.**



11. Script 9. Recolección de direcciones IP que acceden al servidor

Sirve para mapear comportamiento.

```
import re
ips = set()

with open("auth.log") as f:
    for linea in f:
        m = re.search(r"from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)", linea)
        if m:
            ips.add(m.group(1))

print("IPs únicas:", ips)
```

y con este script podemos ver cuáles son esas direcciones IP que acceden a nuestro servidor:

```
ozone@ubuntu-server:~/soc10$ sudo python3 ip_acceden_servidor.py
IPs únicas: {'192.168.0.33', '192.168.0.38'}
```

Si te aparece una dirección IP que no reconoces entonces puedes investigarla.

12. Script 10: Detectar patrones de escaneo (nmap, masscan, etc.)

Los escaneos dejan trazas en los logs:

```
import re
from collections import defaultdict

ruta = "/var/log/kern.log"

patrones_flags = {
    "SYN scan": re.compile(r"Flags \[S\]"),
    "NULL scan": re.compile(r"Flags \[\]\"),
    "FIN scan": re.compile(r"Flags \[F\]"),
    "XMAS scan": re.compile(r"Flags \[(F|P|U|R|S|A){3,}\]\"),
}

# Contadores
puertos_por_ip = defaultdict(set)
paquetes_por_ip = defaultdict(int)
fingerprint_nmap = defaultdict(int)

pat_fingerprint = re.compile(r"TTL=(\d+).*Window=(\d+)")

with open(ruta, "r") as f:
    for linea in f:

        # Obtener IP origen
        m = re.search(r"SRC=([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)", linea)
        if not m:
            continue
        ip = m.group(1)

        paquetes_por_ip[ip] += 1

        # Deteccion de tipos de escaneo por flags
        for tipo, patron in patrones_flags.items():
            if patron.search(linea):
                print(f"[{tipo}] desde {ip}")

        # Deteccion de numero de puertos tocados
        m2 = re.search(r"DPT=([0-9]+)", linea)
        if m2:
            puerto = int(m2.group(1))
```

```

        puertos_por_ip[ip].add(puerto)

# Deteccion de fingerprint Nmap
fp = pat_fingerprint.search(linea)
if fp:
    ttl, win = fp.groups()
    key = (ttl, win)
    fingerprint_nmap[key] += 1

# Reporte final
print("\n--- RESUMEN ---")

for ip, puertos in puertos_por_ip.items():
    if len(puertos) > 20:
        print(f"[Scan masivo] {ip} toco {len(puertos)} puertos")

for ip, total in paquetes_por_ip.items():
    if total > 100:
        print(f"[Actividad sospechosa] {ip} genero {total} paquetes")

for (ttl, win), count in fingerprint_nmap.items():
    if count > 30:
        print(f"[Patron Nmap] TTL={ttl} Window={win} repetido {count} veces")

```

Cuando lo ejecutamos podemos ver el resultado:

```

ozone@ubuntu-server:~/soc10$ sudo python3 detecta_scan.py
--- RESUMEN ---
[Actividad sospechosa] 192.168.0.33 genero 208 paquetes

```

Advertencia. Si no funciona es porque debes hacer un cambio en iptables.

Para que el script funcione, necesitamos usar **iptables** para activar el "modo chismoso" en el servidor. Tenemos que agregar una regla que le diga al sistema: "Cada vez que bloquee o recibas un paquete sospechoso, antes de hacer nada, escribe sus detalles en el log del kernel".

El comando en iptables suele verse así:

```
iptables -A INPUT -p tcp -j LOG --log-prefix "IPTABLES: "
```

En resumen:

- **Sin el cambio en iptables:** El servidor bloquea los escaneos en silencio. /var/log/kern.log se queda vacío y el script no detecta nada.
- **Con el cambio en iptables:** Cada intento de escaneo deja una huella escrita (Flags, TTL, Puerto). El script lee esa huella y activa las alertas.

Importante: Esto puede ser útil para un laboratorio como este en el que estamos entendiendo las herramientas o un servidor pequeño con poco tráfico debido a que si escribimos toda esa información, la carga puede terminar por hacer caer el servidor. Para producción, es obligatorio usar filtros de límite (limit) para no suicidar el rendimiento del equipo.

Dicho lo anterior, para poder ejecutar nuestro script en nuestro laboratorio haremos lo siguiente:

12.1 Activar logs de paquetes en tiempo real

Sin esto, NINGUN script de detección funcionara. Necesitamos agregar una regla que loguee **TODOS** los paquetes entrantes que no estén permitidos y que incluya **flags, origen, destino y puerto**

12.1.1 Activar logging en IPTABLES

Ejecutamos:

```
sudo iptables -A INPUT -m limit --limit 5/second -j LOG --log-prefix "SCAN: "
```

Esto agrega:

- Logueo de paquetes entrantes
- Con prefijo "SCAN: "
- Max 5 por segundo (para no llenar el disco)

Ver en tiempo real si funciona con:

```
sudo tail -f /var/log/kern.log
```

Ahora desde Kali:

```
nmap -sS 192.168.0.35
```

Deberíamos ver líneas como:

```
SCAN: IN=eth0 OUT= MAC=... SRC=192.168.0.50 DST=192.168.0.35 LEN=60 TOS=0x00  
PREC=0x00 TTL=64 ID=12345 DF PROTO=TCP SPT=50000 DPT=22 WINDOW=1024 RES=0x00  
SYN URGP=0
```

Si ves esto entonces nuestro script ya empezará a detectar todo.

12.1.2 Hacer persistente la regla

Instalamos:

```
sudo apt install iptables-persistent
```

Guardamos las reglas:

```
sudo netfilter-persistent save
```

12.1.3 Volver a correr el script

```
sudo python3 detecta_scan.py
```

Y desde Kali:

```
nmap -sV 192.168.0.35
```

Ahora debería imprimir:

```
[SYN scan] desde 192.168.0.50  
[Scan masivo] 192.168.0.50 toco 30 puertos  
[Actividad sospechosa] 192.168.0.50 genero 150 paquetes  
[Patron Nmap] TTL=64 Window=1024 repetido 40 veces
```

Recordar que esto es solo para entender las herramientas y no debiese ser utilizado en un servidor en producción.

13. Script 11: Monitoreo de uso de red en tiempo real

En Linux tenemos el comando **tail -f** que sirve para mostrar las últimas líneas de un archivo. A partir de esta lógica podemos automatizar un script para que nos haga un monitoreo del uso de red en tiempo real.

```
nano alertas_tiempo_real.py
```

```
import re
from collections import defaultdict

ruta = "/var/log/kern.log"

patrones_flags = {
    "SYN scan": re.compile(r"Flags \[S\]"),
    "NULL scan": re.compile(r"Flags \[\]\"),
    "FIN scan": re.compile(r"Flags \[F\]"),
    "XMAS scan": re.compile(r"Flags \[(F|P|U|R|S|A){3,}\]"),
}

# Contadores
puertos_por_ip = defaultdict(set)
paquetes_por_ip = defaultdict(int)
fingerprint_nmap = defaultdict(int)

pat_fingerprint = re.compile(r"TTL=(\d+).*Window=(\d+)")

with open(ruta, "r") as f:
    for linea in f:

        # Obtener IP origen
        m = re.search(r"SRC=([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)", linea)
        if not m:
            continue
        ip = m.group(1)

        paquetes_por_ip[ip] += 1

        # Detección de tipos de escaneo por flags
        for tipo, patron in patrones_flags.items():
            if patron.search(linea):
                print(f"[{tipo}] desde {ip}")
```

```

# Deteccion de numero de puertos tocados
m2 = re.search(r"DPT=([0-9]+)", linea)
if m2:
    puerto = int(m2.group(1))
    puertos_por_ip[ip].add(puerto)

# Deteccion de fingerprint Nmap
fp = pat_fingerprint.search(linea)
if fp:
    ttl, win = fp.groups()
    key = (ttl, win)
    fingerprint_nmap[key] += 1

# Reporte final
print("\n--- RESUMEN ---")

for ip, puertos in puertos_por_ip.items():
    if len(puertos) > 20:
        print(f"[Scan masivo] {ip} toco {len(puertos)} puertos")

for ip, total in paquetes_por_ip.items():
    if total > 100:
        print(f"[Actividad sospechosa] {ip} genero {total} paquetes")

for (ttl, win), count in fingerprint_nmap.items():
    if count > 30:
        print(f"[Patron Nmap] TTL={ttl} Window={win} repetido {count} veces")

```

14. Script 12: Monitoreo de usuarios

Sirve para detectar de inmediato si un atacante intentó crear una cuenta persistente para quedarse en el sistema o si alteró las contraseñas del servidor.

```
nano monitoreo_usuarios.py
```

```
import re
import hashlib

auth_log = "/var/log/auth.log"

# =====
# 1. Detectar creacion de usuarios
# =====
patron_creacion = re.compile(
    r"user(?:add)?\[\d+\]: new user: name=([a-zA-Z0-9_-]+)"
)
# =====
# 2. Detectar cambios de contrasena
# =====
patron_passwd = re.compile(
    r"passwd\[\d+\]: password changed for ([a-zA-Z0-9_-]+)"
)
# =====
# 3. Detectar bloqueo / desbloqueo
# =====
patron_bloqueo = re.compile(
    r"passwd\[\d+\]: account (locked|unlocked) for user ([a-zA-Z0-9_-]+)"
)
# =====
# 4. Detectar uso de usermod, userdel, adduser
# =====
patron_user_admin = re.compile(
    r"(usermod|userdel|adduser|useradd)\[\d+\]:"
)
# =====
# 5. Detectar cambios en /etc/passwd y /etc/shadow via hashes
# =====

def hash_archivo(ruta):
    try:
        with open(ruta, "rb") as f:
            return hashlib.sha256(f.read()).hexdigest()
```

```

except:
    return None

hash_passwd_inicial = hash_archivo("/etc/passwd")
hash_shadow_inicial = hash_archivo("/etc/shadow")

# =====
# 6. Analizar el archivo de logs
# =====
print("\n=== EVENTOS DETECTADOS EN auth.log ===\n")

with open(auth_log) as f:
    for linea in f:
        # Creacion de usuarios
        m = patron_creacion.search(linea)
        if m:
            print(f"[Nuevo usuario] {m.group(1)}")
        # Cambio de contraseña
        m = patron_passwd.search(linea)
        if m:
            print(f"[Cambio de contraseña] Usuario: {m.group(1)}")
        # Bloqueo / desbloqueo
        m = patron_bloqueo.search(linea)
        if m:
            accion, usuario = m.groups()
            print(f"[{accion.upper()}] Usuario: {usuario}")
        # Otros comandos de administracion
        if patron_user_admin.search(linea):
            print(f"[Administracion de cuentas] {linea.strip()}")

# =====
# 7. Verificar integridad de archivos
# =====
hash_passwd_actual = hash_archivo("/etc/passwd")
hash_shadow_actual = hash_archivo("/etc/shadow")

print("\n=== VERIFICACION DE INTEGRIDAD ===\n")

if hash_passwd_inicial != hash_passwd_actual:
    print("[Alerta] /etc/passwd fue modificado")

if hash_shadow_inicial != hash_shadow_actual:
    print("[Alerta] /etc/shadow fue modificado")

print("\nAnálisis finalizado.\n")

```


Podemos ver el resultado de su ejecución:

```
ozone@ubuntuuserver:~/soc10$ sudo python3 monitoreo_usuarios.py

=== EVENTOS DETECTADOS EN auth.log ===

[Nuevo usuario] prueba
[Administracion de cuentas] 2026-02-23T16:07:09.934245-03:00 ubuntuuserver useradd
[1637]: new user: name=prueba, UID=1001, GID=1001, home=/home/prueba, shell=/bin/
bash, from=/dev/pts/1

=== VERIFICACION DE INTEGRIDAD ===

Analisis finalizado.
```



15. Script 13: Revisar actividad del firewall (UFW)

Aclaración:

Para que el script funcione debemos tener instalado **ufw** en nuestro servidor Ubuntu; si no lo tienes al igual que yo en este punto, debemos hacer lo siguiente:

15.1 Instalación de UFW

15.1.1 Revisar si UFW está instalado

Ejecutamos:

```
sudo ufw status
```

Casos:

- Si dice:
command not found = no está instalado.
- Si muestra un estado
(active/inactive) = si está instalado.

15.1.2 Verificar si el paquete existe

Ejecutamos:

```
apt list --installed | grep ufw
```

Si aparece algo como:

```
ufw/jammy,now 0.36.1 all [installed]
```

Entonces UFW está instalado.

15.1.3 Ver si el archivo de log existe o si está en otro lado

Ejecutamos:

```
ls -l /var/log | grep ufw
```

Deberíamos ver:

```
ufw.log
```

Si no aparece, aun con UFW instalado, entonces el login está desactivado.

15.1.4 Activar logging de UFW (si esta apagado)

Primero verificamos el nivel:

```
sudo ufw status verbose
```

Buscamos la línea:

```
Logging: on
```

o

```
Logging: off
```

Si esta apagado, lo activamos con:

```
sudo ufw logging on
```

Esto crea y comienza a llenar `/var/log/ufw.log`.

15.1.5 Si UFW no está instalado

Lo instalamos con:

```
sudo apt install ufw -y
```

Lo activamos (si quieres):

```
sudo ufw enable
```

Advertencia: si usas SSH, antes permite el puerto:

```
sudo ufw allow ssh
```

15.1.6 Verificar si UFW está realmente activo en Ubuntu

En Ubuntu Server, ejecutamos:

```
sudo ufw status verbose
```

```

ozone@ubuntuuserver:~/soc10$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

```

15.1.7 Comprobar si apareció el log

En Ubuntu:

```
ls -l /var/log | grep ufw
```

```

ozone@ubuntuuserver:~/soc10$ ls -l /var/log | grep ufw
-rw-r----- 1 syslog adm 3393 Feb 27 16:05 ufw.log

```

Activar logging a nivel más detallado:

```
sudo ufw logging high
```

```

ozone@ubuntuuserver:~/soc10$ sudo ufw logging high
[sudo] password for ozone:
Logging enabled

```

Haremos una prueba con:

```
sudo tail -f /var/log/ufw.log
```

```

ozone@ubuntuuserver:~/soc10$ sudo tail -f /var/log/ufw.log
2026-02-27T16:22:33.446615-03:00 ubuntuuserver kernel: [UFW BLOCK] IN=ens33 OUT= M
AC=00: SRC=192.168.0.38 DST=192.168.0.35 LE
N=60 TOS=0x00 PREC=0x00 TTL=64 ID=17849 DF PROTO=TCP SPT=45572 DPT=9999 WINDOW=64
240 RES=0x00 SYN URGP=0
2026-02-27T16:22:34.470095-03:00 ubuntuuserver kernel: [UFW BLOCK] IN=ens33 OUT= M
AC= SRC=192.168.0.38 DST=192.168.0.35 LE
N=60 TOS=0x00 PREC=0x00 TTL=64 ID=17850 DF PROTO=TCP SPT=45572 DPT=9999 WINDOW=64
240 RES=0x00 SYN URGP=0
2026-02-27T16:22:35.493853-03:00 ubuntuuserver kernel: [UFW BLOCK] IN=ens33 OUT= M
AC= SRC=192.168.0.38 DST=192.168.0.35 LE
N=60 TOS=0x00 PREC=0x00 TTL=64 ID=17851 DF PROTO=TCP SPT=45572 DPT=9999 WINDOW=64
240 RES=0x00 SYN URGP=0

```


15.2 Creación del script para revisar UFW

nano actividad_firewall.py

```
import re
from collections import defaultdict

# Regex mejorada: detecta [UFW BLOCK] o cualquier texto que incluya BLOCK
patron = re.compile(
    r"UFW BLOCK.*SRC=([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+).*DST=([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+).*DPT=(\d+)",
    re.IGNORECASE
)

trafico = defaultdict(lambda: {"puertos": [], "conteo": 0})

# Leer log real
with open("/var/log/ufw.log") as f:
    for linea in f:
        m = patron.search(linea)
        if m:
            src, dst, puerto = m.groups()
            trafico[src]["puertos"].append(int(puerto))
            trafico[src]["conteo"] += 1

# Puertos sensibles
puertos_sensibles = {22, 23, 3389, 445, 21, 3306, 5432}

def es_interna(ip):
    return (
        ip.startswith("10.")
        or ip.startswith("192.168.")
        or ip.startswith("172.16.")
        or ip.startswith("172.17.")
        or ip.startswith("172.18.")
        or ip.startswith("172.19.")
        or ip.startswith("172.20.")
        or ip.startswith("172.21.")
        or ip.startswith("172.22.")
        or ip.startswith("172.23.")
        or ip.startswith("172.24.")
        or ip.startswith("172.25.")
        or ip.startswith("172.26.")
        or ip.startswith("172.27.")
        or ip.startswith("172.28.")
    )
```

```

        or ip.startswith("172.29.")
        or ip.startswith("172.30.")
        or ip.startswith("172.31.")
    )

# Reporte
for ip, datos in trafico.items():
    puertos = datos["puertos"]
    intentos = datos["conteo"]

    print("\nIP:", ip)
    print("Intentos bloqueados:", intentos)
    print("Puertos probados:", sorted(set(puertos)))

    # indicador de escaneo
    if len(set(puertos)) > 8:
        print(">> Posible escaneo de puertos")

    # puertos sensibles
    sensibilidad = set(puertos) & puertos_sensibles
    if sensibilidad:
        print(">> Intentos hacia puertos sensibles:", sensibilidad)

    # trafico interno o externo
    if es_interna(ip):
        print("Tipo: Trafico interno")
    else:
        print("Tipo: Trafico externo")

    # alerta fuerte
    if len(sensibilidad) > 2:
        print(">> Alerta: multiples puertos sensibles atacados")

print("\nAnálisis UFW completado.")

```

cuando lo ejecutamos recibimos:

```
ozone@ubuntuerver:~/soc10$ sudo python3 actividad_firewall.py
```

```
IP: 192.168.0.252  
Intentos bloqueados: 1  
Puertos probados: [5353]  
Tipo: Trafico interno
```

```
IP: 192.168.0.38  
Intentos bloqueados: 26  
Puertos probados: [9999]  
Tipo: Trafico interno
```

```
Analisis UFW completado.
```

```
ozone@ubuntuerver:~/soc10$ |
```



16. Script 14: Monitorear archivos de crontab por cambios

Los atacantes usan cron para persistencia.

```
import os
import subprocess
import hashlib
from datetime import datetime

# Archivo donde se guarda el hash previo
HASH_FILE = "cron_hash.txt"

# Obtener crontab actual en texto
def obtener_crontab():
    try:
        salida = subprocess.check_output(["crontab", "-l"],
stderr=subprocess.STDOUT)
        return salida.decode()
    except subprocess.CalledProcessError:
        return "" # Si el usuario no tiene crontab

# Calcular hash
def calcular_hash(texto):
    return hashlib.sha256(texto.encode()).hexdigest()

# Guardar hash
def guardar_hash(h):
    with open(HASH_FILE, "w") as f:
        f.write(h)

# Leer hash previo
def leer_hash_previo():
    if not os.path.exists(HASH_FILE):
        return None
    with open(HASH_FILE, "r") as f:
        return f.read().strip()

# Main
crontab_actual = obtener_crontab()
hash_actual = calcular_hash(crontab_actual)
hash_previo = leer_hash_previo()

if hash_previo is None:
```



```

print("Primera ejecucion. Guardando estado inicial...")
guardar_hash(hash_actual)
else:
    if hash_actual != hash_previo:
        print("CAMBIO DETECTADO EN CRONTAB!")
        print("Fecha:", datetime.now())
        print("\nContenido nuevo del crontab:\n")
        print(crontab_actual)
        guardar_hash(hash_actual)
    else:
        print("Sin cambios en crontab.")

```

La primera vez que lo ejecutemos nos indicará:

```

ozone@ubuntuerver:~/soc10$ sudo python3 crontab_cambios.py
Primera ejecucion. Guardando estado inicial...

```

Si hacemos una prueba agregando algo con `crontab -e`

```

ozone@ubuntuerver:~/soc10$ crontab -e
no crontab for ozone - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab

```

```
* * * * * echo "Hola" >> /tmp/test.txt
```

```
GNU nano 7.2 /tmp/crontab.xaguqn/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
* * * * * echo "Hola" >> /tmp/test.txt
```

Y cuando ejecutamos nos dice lo siguiente:

```
ozone@ubuntuuserver:~/soc10$ python3 crontab_cambios.py
CAMBIO DETECTADO EN CRONTAB!
Fecha: 2026-02-27 17:00:09.639599

Contenido nuevo del crontab:

# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
* * * * * echo "Hola" >> /tmp/test.txt

Traceback (most recent call last):
  File "/home/ozone/soc10/crontab_cambios.py", line 47, in <module>
    guardar_hash(hash_actual)
  File "/home/ozone/soc10/crontab_cambios.py", line 23, in guardar_hash
    with open(HASH_FILE, "w") as f:
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
PermissionError: [Errno 13] Permission denied: 'cron_hash.txt'
```

Aclaración: Cuando ejecute el script por primera vez lo hice con sudo (revisar las imágenes) que fue cuando guardo el estado del crontab inicial. Cuando lo ejecute por segunda vez nuevamente con sudo, el script no arrojó nada por que al estar con sudo estaba buscando en el crontab de ROOT y no en el del usuario OZONE... por

este motivo se puede ver que al ejecutar el script sin el sudo como lo muestra la imagen si funciona. Pero nos sigue dejando un error al final

```
* * * * * echo "Hola" >> /tmp/test.txt

Traceback (most recent call last):
  File "/home/ozone/soc10/crontab_cambios.py", line 47, in <module>
    guardar_hash(hash_actual)
  File "/home/ozone/soc10/crontab_cambios.py", line 23, in guardar_hash
    with open(HASH_FILE, "w") as f:
        ^^^^^^^^^^^^^^^^^^^^^^^^^
PermissionError: [Errno 13] Permission denied: 'cron_hash.txt'
```

Respuesta de la inteligencia artificial:

*El archivo **cron_hash.txt** fue creado **antes** cuando ejecutaste el script con sudo, así que quedo con propietario ROOT, no ozone.*

Por eso ahora, cuando el script corre como usuario normal, te dice:

PermissionError: [Errno 13] Permission denied: 'cron_hash.txt'

La solución:

Ejecuta esto desde tu carpeta ~/soc10:

```
sudo chown ozone:ozone cron_hash.txt
```

Eso cambia el propietario del archivo al usuario ozone. Luego ejecuta de nuevo:

```
python3 crontab_cambios.py
```

Debe funcionar sin errores.

Como muestra la imagen yo edite el archivo crontab con `crontab -e` nuevamente y elimine la línea de prueba que había agregado previamente

```
* * * * * echo "Hola" >> /tmp/test.txt
```

y este es el cambio que registra y ahora vemos que ya no tenemos las líneas que indican el error al final de la salida.

```
ozone@ubuntu-server:~/soc10$ python3 crontab_cambios.py
CAMBIO DETECTADO EN CRONTAB!
Fecha: 2026-02-27 17:06:41.256349

Contenido nuevo del crontab:

# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
```

Como siempre digo, la inteligencia artificial es para mi una herramienta más que un medio para hacer todo, sobre todo cuando estamos aprendiendo es importante entender nuestros errores. Por este motivo quise indagar un poco mas en este error y dejo la transcripción a continuación de las respuestas.

¿Por qué ocurrió el error?

1. Ejecutaste el script con sudo
2. El archivo **cron_hash.txt** se creó con permisos de root
3. Luego ejecutaste sin sudo
4. El usuario ozone no puede modificar un archivo que pertenece a root
5. Error de permiso

Si quieres borrar el hash y empezar de cero

También puedes simplemente eliminar el archivo:

```
rm cron_hash.txt
```

Y el script lo creará de nuevo.

¿Y que sería el archivo cron_hast.txt?

El archivo **cron_hash.txt** es simplemente un **archivo de control** que usa tu script para recordar cómo estaba el crontab la última vez que lo reviso.

- No es un archivo del sistema.
- No lo crea Ubuntu.
- No lo usa cron.

Lo crea tu script.

¿Para qué sirve exactamente?

Cada vez que el script lee tu crontab, calcula un **hash SHA256** del contenido. Ese hash es una huella digital única del crontab.

Ejemplo:

- Si el crontab tiene 10 líneas → se genera un hash.
- Si cambias solo una letra → el hash cambia completamente.

El script compara:

hash actual vs hash guardado en `cron_hash.txt`

Si son iguales:

→ No hubo cambios.

Si son diferentes:

→ Detecta un cambio y te lo avisa.

Después de avisar, actualiza el hash almacenado en **`cron_hash.txt`** para que la próxima vez tenga con que comparar.

¿Que contiene `cron_hash.txt` por dentro?

Contiene solo una línea como esta:

`e4f2c18db239d98a7c3dff0cb5bb70b996b1a0e3dc421b6a0d2a7f21b5e0d322`

Eso es la huella del crontab. Nada más.

¿Qué sucede si lo borras?

Nada malo. Al contrario:

`rm cron_hash.txt`

La próxima vez que ejecutes el script dirá:

Primera ejecución. Guardando estado inicial...

Y lo creará de nuevo.

Ahora me quedo claro el error y entiendo mucho mas el crontab.

17. Script 15: Buscar palabras clave en logs (IOCs básicos)

Este script actúa como un **detector de Indicadores de Compromiso (IoCs)** automatizado. Su importancia radica en dos puntos clave:

- **Monitorea el ciclo completo de un ataque:** Rastrea desde el intento inicial (usuarios inválidos y contraseñas fallidas) hasta el peor escenario posible en ciberseguridad: que un atacante logre convertirse en administrador (root inició sesión o fallo en su).
- **Simplifica la cacería de amenazas (*Threat Hunting*):** En lugar de buscar manualmente entre millones de líneas confusas, el script traduce los códigos complejos del sistema a alertas humanas claras y críticas en tiempo real.

Básicamente, es el encargado de avisarnos cuando pasamos de estar "siendo atacado" a estar "**totalmente hackeados**".

Creemos el script con:

```
nano palabras_clave_logs.py
```

```
ioc_list = [
    ("Failed password", "SSH password incorrecta"),
    ("Invalid user", "Usuario inexistente intentando acceder"),
    ("error", "Error generico"),
    ("session opened for user root", "Root inicio sesion"),
    ("session closed for user root", "Root cerro sesion"),
    ("su: authentication failure", "Fallo en su (cambio a root)"),
]

with open("auth.log") as f:
    for linea in f:
        for ioc, descripcion in ioc_list:
            if ioc.lower() in linea.lower():
                print(f"[CRITICO] {descripcion} -> {linea.strip()}")
```


ejecutamos y vemos como nos acusa entre otras cosas inicios de sesión root... si somos el usuario root y no hemos iniciado sesión es un momento para preocuparse y revisar la actividad.

```
ozone@ubuntuuserver:~/soc10$ python3 palabras_clave_logs.py
[CRITICO] Root inicio sesion -> 2026-02-23T16:05:00.645138-03:00 ubuntuuserver CRO
N[1381]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=
0)
[CRITICO] Root cerro sesion -> 2026-02-23T16:05:00.654270-03:00 ubuntuuserver CRON
[1381]: pam_unix(cron:session): session closed for user root
[CRITICO] Root inicio sesion -> 2026-02-23T16:07:09.739429-03:00 ubuntuuserver sud
o: pam_unix(sudo:session): session opened for user root(uid=0) by ozone(uid=1000)
[CRITICO] Root cerro sesion -> 2026-02-23T16:08:18.697290-03:00 ubuntuuserver sudo
: pam_unix(sudo:session): session closed for user root
[CRITICO] Root inicio sesion -> 2026-02-23T16:12:49.947044-03:00 ubuntuuserver sud
o: pam_unix(sudo:session): session opened for user root(uid=0) by ozone(uid=1000)
[CRITICO] Root inicio sesion -> 2026-02-23T16:15:01.675707-03:00 ubuntuuserver CRO
N[1697]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=
0)
```



18. Script 16: SOC completo

SOC completo; análisis defensivo de seguridad para Linux en un solo archivo Python, sin dependencias ni instalación.

Llega a cualquier servidor, corre en segundos, y dice qué tan comprometido está.

OJO: Es una fotografía del momento, no monitoreo en tiempo real. Todo lo que detecta refleja el estado del sistema en el instante que la corres.

Diseñada para el escenario donde un analista llega a un servidor potencialmente comprometido, sin acceso a herramientas de monitoreo y sin poder instalar nada; solo copia el archivo, lo ejecuta, y tiene un diagnóstico estructurado en segundos.

```
#!/usr/bin/env python3
# =====
#   SOC PUNTUAL v3.0 – Análisis Defensivo Profundo
#   Linux (Debian/Ubuntu) – Un solo archivo, cero dependencias
#   Uso:          sudo python3 soc.py
#   Reporte:      sudo python3 soc.py --report [--html] [--json]
#   Cron:         sudo python3 soc.py --report --html --no-color
# =====

import re, os, sys, json, hashlib, subprocess, shutil
import datetime, io, argparse, stat, socket, glob
from collections import defaultdict
from concurrent.futures import ThreadPoolExecutor, TimeoutError as
FuturesTimeout

# -----
#   CONFIGURACIÓN
# -----

VERSION          = "3.0"
LOG_CANDIDATOS   = ["/var/log/auth.log", "/var/log/secure", "/var/log/syslog"]
BASELINE_FILE    = "baseline_hashes.json"
HORA_INICIO      = 6
HORA_FIN         = 22
CMD_TIMEOUT      = 10

PROCS_SOSPECHOSOS = {
    "nc", "ncat", "nmap", "socat", "msfconsole", "netcat",
```

```

    "xmrigr", "minerd", "cgminer", "tcpdump", "tshark",
    "responder", "hydra", "john", "hashcat", "aircrack-ng",
}

PUERTOS_SOSPECHOSOS = {
    4444, 1337, 31337, 6666, 6667, 6668, 6669, 9999,
    5555, 8888, 2222, 12345, 1234, 54321,
}

ARCHIVOS_CRITICOS = [
    "/etc/passwd", "/etc/shadow", "/etc/group", "/etc/sudoers",
    "/etc/crontab", "/etc/hosts", "/etc/ssh/sshd_config",
    "/bin/bash", "/usr/bin/sudo", "/usr/bin/python3",
    "/usr/bin/curl", "/usr/bin/wget", "/usr/bin/awk",
    "/usr/bin/find", "/usr/bin/chmod", "/usr/bin/chown",
]

SHELLS_VALIDAS =
{" /bin/bash", "/bin/sh", "/usr/bin/zsh", "/usr/bin/fish", "/bin/zsh"}

SUID_ESPERADOS = {
    "/usr/bin/sudo", "/usr/bin/su", "/usr/bin/passwd", "/usr/bin/newgrp",
    "/usr/bin/gpasswd", "/usr/bin/chfn", "/usr/bin/chsh", "/bin/su",
    "/bin/mount", "/bin/umount", "/usr/bin/pkexec",
    "/usr/lib/openssh/ssh-keysign",
    "/usr/lib/dbus-1.0/dbus-daemon-launch-helper",
    "/sbin/unix_chkpwd", "/usr/sbin/pam_timestamp_check",
    "/usr/bin/crontab", "/usr/bin/at", "/usr/bin/wall",
    "/usr/bin/write", "/usr/bin/ssh-agent",
}

# -----
#   SISTEMA DE FINDINGS
# -----

SEV_CRITICAL = "CRÍTICO"
SEV_HIGH     = "ALTO"
SEV_MEDIUM   = "MEDIO"
SEV_INFO     = "INFO"
SEV_OK       = "OK"

SEV_ORDER = {SEV_CRITICAL: 0, SEV_HIGH: 1, SEV_MEDIUM: 2, SEV_INFO: 3,
SEV_OK: 4}

class Finding:
    def __init__(self, severity, module, title, detail=""):

```

```

        self.severity = severity
        self.module   = module
        self.title     = title
        self.detail    = detail
        self.ts        = datetime.datetime.now().isoformat()

FINDINGS = []

def finding(severity, module, title, detail=""):
    f = Finding(severity, module, title, detail)
    FINDINGS.append(f)
    return f

# -----
# COLORES ANSI
# -----

class C:
    CRIT = "\033[1;31m"
    HIGH = "\033[91m"
    MED  = "\033[93m"
    INFO = "\033[96m"
    OK   = "\033[92m"
    BOLD = "\033[1m"
    DIM  = "\033[2m"
    RESET = "\033[0m"
    SEP  = "\033[90m"

USE_COLOR = True

def col(code, text):
    return f"{code}{text}{C.RESET}" if USE_COLOR else text

SEV_COLOR = {
    SEV_CRITICAL: C.CRIT, SEV_HIGH: C.HIGH,
    SEV_MEDIUM: C.MED, SEV_INFO: C.INFO, SEV_OK: C.OK,
}

def _print_finding(f):
    color = SEV_COLOR.get(f.severity, C.RESET)
    tag   = col(color, f"[{f.severity:8s}]")
    print(f" {tag} {f.title}")
    if f.detail:
        for line in f.detail.splitlines():
            print(f" {col(C.DIM, line)}")

```



```

def sep(title=""):
    line = col(C.SEP, "-" * 55)
    if title:
        print(f"\n{line} {col(C.BOLD, title)}")
    else:
        print(f"\n{line}")

# -----
# UTILIDADES
# -----

def _detectar_log():
    for path in LOG_CANDIDATOS:
        if os.path.exists(path) and os.access(path, os.R_OK):
            return ("file", path)
    if shutil.which("journalctl"):
        return ("journal", None)
    return (None, None)

def _leer_logs():
    fuente, path = _detectar_log()
    if fuente == "file":
        try:
            with open(path, "r", errors="replace") as f:
                return f.readlines()
        except Exception:
            return []
    elif fuente == "journal":
        salida = _ejecutar(["journalctl", "-u", "ssh", "--no-pager",
                           "--since", "7 days ago", "-o", "short-iso"])
        return salida.splitlines(keepends=True)
    return []

def _parsear_fecha(linea):
    m = re.match(r"(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2})", linea)
    if m:
        try:
            return datetime.datetime.fromisoformat(m.group(1))
        except ValueError:
            pass
    partes = linea.split()
    if len(partes) >= 3:
        try:
            anio = datetime.datetime.now().year
            return datetime.datetime.strptime(
                f"{partes[0]} {partes[1].zfill(2)} {partes[2]} {anio}",

```

```

        "%b %d %H:%M:%S %Y"
    )
    except (ValueError, IndexError):
        pass
    return None

def _ejecutar(cmd, timeout=CMD_TIMEOUT):
    def _run():
        return subprocess.check_output(cmd,
stderr=subprocess.DEVNULL).decode(errors="replace")
    try:
        with ThreadPoolExecutor(max_workers=1) as ex:
            return ex.submit(_run).result(timeout=timeout)
    except FuturesTimeout:
        return ""
    except subprocess.CalledProcessError as e:
        return e.output.decode(errors="replace") if e.output else ""
    except FileNotFoundError:
        return ""
    except Exception:
        return ""

def _hash_archivo(path):
    h = hashlib.sha256()
    try:
        with open(path, "rb") as f:
            for bloque in iter(lambda: f.read(65536), b''):
                h.update(bloque)
            return h.hexdigest()
    except FileNotFoundError:
        return None
    except PermissionError:
        return "SIN_PERMISO"

def _capturar_salida(func):
    buf = io.StringIO()
    orig = sys.stdout
    sys.stdout = buf
    try:
        func()
    finally:
        sys.stdout = orig
    return buf.getvalue()

def _leer_archivo(path):

```

```

try:
    with open(path, "r", errors="replace") as f:
        return f.read()
except (FileNotFoundError, PermissionError):
    return None

def _listar_usuarios_reales():
    """Retorna lista de (usuario, home, shell) con uid >= 1000 o root."""
    usuarios = []
    try:
        with open("/etc/passwd") as f:
            for linea in f:
                partes = linea.strip().split(":")
                if len(partes) < 7:
                    continue
                nombre, uid, home, shell = partes[0], partes[2], partes[5],
partes[6]
                try:
                    uid_int = int(uid)
                except ValueError:
                    continue
                if uid_int == 0 or uid_int >= 1000:
                    usuarios.append((nombre, home, shell))
    except Exception:
        pass
    return usuarios

# -----
# MÓDULO 1: LOGS SSH / AUTH
# -----

def mod_ssh_logs():
    MOD = "SSH/Auth"
    sep("LOGS SSH Y AUTENTICACIÓN")
    lineas = _leer_logs()

    if not lineas:
        _print_finding(finding(SEV_INFO, MOD, "No se encontraron logs de
autenticación"))
        return

    # SSH fallidos
    fallidos = [l.strip() for l in lineas if "Failed password" in l]
    if fallidos:
        _print_finding(finding(SEV_HIGH, MOD,
f"{len(fallidos)} intentos SSH fallidos",

```

```

        "\n".join(fallidos[-10:]) + ("\n ..." if len(fallidos) > 10
else "")))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin intentos SSH fallidos"))

# Fuerza bruta por IP
patron = re.compile(r"Failed password.*from\s+([\d\.]{7,15})")
ips = defaultdict(int)
for l in lineas:
    m = patron.search(l)
    if m:
        ips[m.group(1)] += 1
sospechosas = {ip: c for ip, c in ips.items() if c >= 5}
if sospechosas:
    detalle = "\n".join(f"{ip:20s} → {cnt} intentos"
                        for ip, cnt in sorted(sospechosas.items(),
key=lambda x: -x[1]))
    sev = SEV_CRITICAL if any(c >= 20 for c in sospechosas.values())
else SEV_HIGH
    _print_finding(finding(sev, MOD,
        f"{len(sospechosas)} IP(s) con posible fuerza bruta (≥5
intentos)", detalle))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin IPs con fuerza bruta"))

# Accesos fuera de horario
accesos_nocturnos = []
for l in lineas:
    if any(k in l for k in ("Accepted password", "Accepted
publickey", "session opened")):
        t = _parsear_fecha(l)
        if t and (t.hour < HORA_INICIO or t.hour >= HORA_FIN):
            accesos_nocturnos.append(l.strip())
if accesos_nocturnos:
    _print_finding(finding(SEV_HIGH, MOD,
        f"{len(accesos_nocturnos)} acceso(s) fuera de horario
({HORA_INICIO}h-({HORA_FIN}h)",
        "\n".join(accesos_nocturnos)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin accesos fuera de horario"))

# Creación de usuarios
nuevos = [l.strip() for l in lineas
            if "useradd" in l or "new user" in l.lower() or "adduser" in
1]

```



```

    if nuevos:
        _print_finding(finding(SEV_HIGH, MOD,
                                f"{len(nuevos)} evento(s) de creación de usuario",
                                "\n".join(nuevos)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin creación de usuarios
detectada"))

# Posibles escaneos
scan_pats = ["Did not receive identification string", "Invalid user",
             "Connection closed by invalid user", "Bad protocol version",
             "Unable to negotiate"]
scans = [l.strip() for l in lineas if any(p in l for p in scan_pats)]
if scans:
    _print_finding(finding(SEV_MEDIUM, MOD,
                            f"{len(scans)} evento(s) de posible escaneo", "\n".join(scans[-
10:]))))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin indicios de escaneo de
red"))

# Sudo
sudo_eventos = [l.strip() for l in lineas if "sudo:" in l]
if sudo_eventos:
    _print_finding(finding(SEV_INFO, MOD,
                            f"{len(sudo_eventos)} evento(s) de sudo",
                            "\n".join(sudo_eventos[-15:]))))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin uso de sudo en logs"))

# -----
#  MÓDULO 2: PROCESOS Y RED
#  -----

def mod_procesos_red():
    MOD = "Procesos/Red"
    sep("PROCESOS Y CONEXIONES DE RED")

    # Procesos sospechosos
    salida_ps = _ejecutar(["ps", "auxww"])
    procs_malos = []
    for linea in salida_ps.splitlines()[1:]:
        partes = linea.split(None, 10)
        if len(partes) < 11:
            continue
        cmd_path = partes[10].split()[0]

```

```

cmd_base = os.path.basename(cmd_path).lower()
if cmd_base in PROCS_SOSPECHOSOS:
    procs_malos.append(linea.strip())
elif cmd_base in ("bash", "sh", "python", "python3", "perl", "ruby"):
    ruta = cmd_path.lstrip("-")
    if ruta and not
ruta.startswith(("bin/", "usr/bin/", "usr/local/bin/", "usr/sbin/")):
    procs_malos.append(f"[shell en ruta inusual]
{linea.strip()}")

if procs_malos:
    _print_finding(finding(SEV_CRITICAL, MOD,
        f"{len(procs_malos)} proceso(s) sospechoso(s)",
"\n".join(procs_malos)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin procesos sospechosos"))

#Puertos en escucha
if shutil.which("ss"):
    salida_ss = _ejecutar(["ss", "-tulpn"])
elif shutil.which("netstat"):
    salida_ss = _ejecutar(["netstat", "-tulpn"])
else:
    salida_ss = ""

puertos_malos = []
for linea in salida_ss.splitlines():
    for p in PUERTOS_SOSPECHOSOS:
        if f":{p}" in linea:
            puertos_malos.append(f"Puerto {p}: {linea.strip()}")
if puertos_malos:
    _print_finding(finding(SEV_CRITICAL, MOD,
        f"{len(puertos_malos)} puerto(s) sospechoso(s) en escucha",
"\n".join(puertos_malos)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin puertos sospechosos en
escucha"))

# Todos los puertos en escucha (info)
if salida_ss:
    escuchando = [l for l in salida_ss.splitlines()
        if "LISTEN" in l or "udp" in l.lower()]
    _print_finding(finding(SEV_INFO, MOD,
        f"{len(escuchando)} servicio(s) en escucha",
"\n".join(escuchando)))

```

```

# Conexiones establecidas
salida_conn = _ejecutar(["ss", "-tunp"])
conn_malas, conn_normales = [], []
for linea in salida_conn.splitlines():
    if "ESTAB" not in linea:
        continue
    partes = linea.split()
    if len(partes) < 6:
        continue
    remoto = partes[5]
    try:
        p_remoto = int(remoto.rsplit(":", 1)[-1])
        if p_remoto in PUERTOS_SOSPECHOSOS:
            conn_malas.append(f"[PUERTO SOSPECHOSO {p_remoto}]
{linea.strip()}")
        else:
            conn_normales.append(linea.strip())
    except ValueError:
        conn_normales.append(linea.strip())

if conn_malas:
    _print_finding(finding(SEV_CRITICAL, MOD,
        f"{len(conn_malas)} conexión(es) a puertos sospechosos",
        "\n".join(conn_malas)))
if conn_normales:
    _print_finding(finding(SEV_INFO, MOD,
        f"{len(conn_normales)} conexión(es) establecida(s)",
        "\n".join(conn_normales)))
if not conn_malas and not conn_normales:
    _print_finding(finding(SEV_OK, MOD, "Sin conexiones activas"))

# Interfaces en modo promiscuo
salida_ip = _ejecutar(["ip", "link", "show"])
promiscuas = [l for l in salida_ip.splitlines() if "PROMISC" in l]
if promiscuas:
    _print_finding(finding(SEV_HIGH, MOD,
        "Interfaz(es) en modo PROMISCO (posible sniffer)",
        "\n".join(promiscuas)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin interfaces en modo
promiscuo"))

# Reglas iptables
if shutil.which("iptables"):

```

```

        salida_ipt = _ejecutar(["iptables", "-L", "-n", "-v"], timeout=5)
        lineas_reglas = [l for l in salida_ipt.splitlines()
                        if l.strip() and not l.startswith("Chain") and not
l.startswith("target")]
        _print_finding(finding(SEV_INFO, MOD,
        f"Reglas iptables ({len(lineas_reglas)} entradas)",
        salida_ipt[:800] if salida_ipt else "Sin reglas o sin
permisos"))

# Crontabs
rutas_cron = ["/etc/crontab", "/etc/cron.d/", "/var/spool/cron/crontabs/",
              "/etc/cron.hourly/", "/etc/cron.daily/"]
entradas_cron = []
for ruta in rutas_cron:
    if os.path.isfile(ruta):
        c = _leer_archivo(ruta)
        if c:
            for linea in c.splitlines():
                if linea.strip() and not linea.startswith("#"):
                    entradas_cron.append(f"[{ruta}] {linea.strip()}")
    elif os.path.isdir(ruta):
        try:
            for arch in sorted(os.listdir(ruta)):
                full = os.path.join(ruta, arch)
                c = _leer_archivo(full)
                if c:
                    for linea in c.splitlines():
                        if linea.strip() and not linea.startswith("#"):
                            entradas_cron.append(f"[{full}]
{linea.strip()}")
        except PermissionError:
            pass

# Crontabs de usuarios reales
for usuario, home, _ in _listar_usuarios_reales():
    cron_u = _ejecutar(["crontab", "-l", "-u", usuario])
    if cron_u and "no crontab" not in cron_u.lower():
        for linea in cron_u.splitlines():
            if linea.strip() and not linea.startswith("#"):
                entradas_cron.append(f"[crontab:{usuario}]
{linea.strip()}")

if entradas_cron:
    _print_finding(finding(SEV_INFO, MOD,
        f"{len(entradas_cron)} tarea(s) programada(s) en cron",

```



```

        "\n".join(entradas_cron)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin tareas cron activas"))

# -----
#  MÓDULO 3: USUARIOS Y PRIVILEGIOS
#  -----
def mod_usuarios_privilegios():
    MOD = "Usuarios/Privilegios"
    sep("USUARIOS Y PRIVILEGIOS")

    # Análisis /etc/passwd
    dup_uid = defaultdict(list)
    uid_cero_noroot = []
    sistema_con_shell = []
    try:
        with open("/etc/passwd") as f:
            for linea in f:
                linea = linea.strip()
                if not linea or linea.startswith("#"):
                    continue
                partes = linea.split(":")
                if len(partes) < 7:
                    continue
                nombre, uid, shell = partes[0], partes[2], partes[6]
                try:
                    uid_int = int(uid)
                except ValueError:
                    continue
                dup_uid[uid].append(nombre)
                if uid_int == 0 and nombre != "root":
                    uid_cero_noroot.append(f"{nombre} (uid=0)")
                if shell in SHELLS_VALIDAS and 0 < uid_int < 1000:
                    sistema_con_shell.append(f"{nombre} (uid={uid_int})
shell={shell}")
            except PermissionError:
                _print_finding(finding(SEV_INFO, MOD, "Sin permisos para leer
/etc/passwd"))

    if uid_cero_noroot:
        _print_finding(finding(SEV_CRITICAL, MOD,
            "Cuentas con UID 0 que no son root",
            "\n".join(uid_cero_noroot)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin cuentas extra con UID 0"))

```

```

if sistema_con_shell:
    _print_finding(finding(SEV_HIGH, MOD,
        "Cuentas de sistema con shell interactiva",
        "\n".join(sistema_con_shell)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin cuentas de sistema con
shell interactiva"))

dup = {uid: us for uid, us in dup_uid.items() if len(us) > 1}
if dup:
    detalle = "\n".join(f"UID {uid}: {' ', ' '.join(us)}" for uid, us in
dup.items())
    _print_finding(finding(SEV_HIGH, MOD, "UIDs duplicados en
/etc/passwd", detalle))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin UIDs duplicados"))

# SSH authorized_keys de todos los usuarios
claves_encontradas = []
for usuario, home, _ in _listar_usuarios_reales():
    ak_path = os.path.join(home, ".ssh", "authorized_keys")
    contenido = _leer_archivo(ak_path)
    if contenido:
        claves = [l.strip() for l in contenido.splitlines()
            if l.strip() and not l.startswith("#")]
        for clave in claves:
            tipo = clave.split()[0] if clave.split() else "?"
            comentario = clave.split()[-1] if len(clave.split()) >= 3
else ""
        claves_encontradas.append(f"[{usuario}] {tipo} ...
{comentario}")

if claves_encontradas:
    _print_finding(finding(SEV_INFO, MOD,
        f"{len(claves_encontradas)} clave(s) SSH autorizada(s)",
        "\n".join(claves_encontradas)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin authorized_keys
configuradas"))

# SSH known_hosts inusuales
hosts_inusuales = []
for usuario, home, _ in _listar_usuarios_reales():
    kh_path = os.path.join(home, ".ssh", "known_hosts")

```

```

    contenido = _leer_archivo(kh_path)
    if contenido:
        hosts = [l.split()[0] for l in contenido.splitlines()
                  if l.strip() and not l.startswith("#") and l.split()]
        if len(hosts) > 50:
            hosts_inusuales.append(f"[{usuario}] {len(hosts)} hosts
conocidos (¿escaneo previo?)")

    if hosts_inusuales:
        _print_finding(finding(SEV_MEDIUM, MOD,
                                "Usuarios con gran cantidad de known_hosts",
                                "\n".join(hosts_inusuales)))

# Sudoers
sudoers = _leer_archivo("/etc/sudoers")
if sudoers:
    lineas_sudo = [l.strip() for l in sudoers.splitlines()
                   if l.strip() and not l.startswith("#") and "NOPASSWD"
in l]
    if lineas_sudo:
        _print_finding(finding(SEV_HIGH, MOD,
                                f"{len(lineas_sudo)} regla(s) sudo con NOPASSWD",
                                "\n".join(lineas_sudo)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin reglas sudo NOPASSWD"))

# Reglas que dan ALL a usuarios no-root
lineas_all = [l.strip() for l in sudoers.splitlines()
               if l.strip() and not l.startswith("#")
               and "ALL=(ALL)" in l and not l.startswith("%")
               and not l.startswith("root")]
if lineas_all:
    _print_finding(finding(SEV_MEDIUM, MOD,
                            "Usuarios con sudo ALL=(ALL)",
                            "\n".join(lineas_all)))

# Sudoers.d
for arch in glob.glob("/etc/sudoers.d/*"):
    contenido = _leer_archivo(arch)
    if contenido:
        malos = [l.strip() for l in contenido.splitlines()
                  if l.strip() and not l.startswith("#") and "NOPASSWD"
in l]
        if malos:
            _print_finding(finding(SEV_HIGH, MOD,

```

```

        f"NOPASSWD en {arch}", "\n".join(malos)))

# Usuarios conectados ahora
who = _ejecutar(["who"])
if who.strip():
    _print_finding(finding(SEV_INFO, MOD,
        "Usuarios actualmente conectados", who.strip()))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin usuarios conectados
actualmente"))

# Últimos logins
last = _ejecutar(["last", "-n", "15"])
if last.strip():
    _print_finding(finding(SEV_INFO, MOD, "Últimos 15 inicios de
sesión", last.strip()))

# Configuración PAM sospechosa
pam_paths = glob.glob("/etc/pam.d/*")
pam_inusual = []
for p in pam_paths:
    contenido = _leer_archivo(p)
    if contenido:
        for linea in contenido.splitlines():
            if "pam_exec" in linea and not
linea.strip().startswith("#"):
                pam_inusual.append(f"[{p}] {linea.strip()}")
if pam_inusual:
    _print_finding(finding(SEV_HIGH, MOD,
        "Módulos pam_exec detectados (posible backdoor PAM)",
        "\n".join(pam_inusual)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin pam_exec inusuales"))

# -----
# MÓDULO 4: ANÁLISIS FORENSE
# -----
def mod_forense():
    MOD = "Forense"
    sep("ANÁLISIS FORENSE DEL SISTEMA")

    # Scripts/ejecutables en /tmp y /var/tmp
    scripts_tmp = []
    for ruta_tmp in ["/tmp", "/var/tmp", "/dev/shm"]:
        salida = _ejecutar(

```



```

        ["find", ruta_tmp, "-maxdepth", "3", "-type", "f",
         "(", "-executable", "-o",
         "-name", "*.sh", "-o", "-name", "*.py",
         "-o", "-name", "*.pl", "-o", "-name", "*.elf", ")"],
        timeout=10
    )
    for arch in salida.splitlines():
        arch = arch.strip()
        if arch:
            scripts_tmp.append(arch)
    if scripts_tmp:
        _print_finding(finding(SEV_CRITICAL, MOD,
                                f"{len(scripts_tmp)} archivo(s) ejecutable(s)/script(s) en /tmp
o /var/tmp",
                                "\n".join(scripts_tmp)))
    else:
        _print_finding(finding(SEV_OK, MOD, "Sin scripts/ejecutables en
/tmp"))

# Directorios ocultos en homes
ocultos = []
for usuario, home, _ in _listar_usuarios_reales():
    try:
        for entry in os.scandir(home):
            if entry.name.startswith(".") and entry.is_dir():
                nombre = entry.name.lower()
                # Directorios ocultos inusuales (no los típicos de
config)

                OCULTOS_NORMALES = {
                    ".ssh", ".config", ".local", ".cache", ".gnupg",
                    ".bash_history", ".profile", ".bashrc", ".bash_logout",
                    ".mozilla", ".thunderbird", ".git", ".vscode",
                }
                if entry.name not in OCULTOS_NORMALES:
                    ocultos.append(f"[{usuario}] {entry.path}")
    except PermissionError:
        pass
if ocultos:
    _print_finding(finding(SEV_MEDIUM, MOD,
                            f"{len(ocultos)} directorio(s) oculto(s) inusual(es) en homes",
                            "\n".join(ocultos)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin directorios ocultos
inusuales en homes"))

```

```

# Archivos modificados en últimas 24h en rutas críticas
rutas_criticas = ["/etc", "/bin", "/usr/bin", "/sbin", "/usr/sbin",
                  "/lib", "/usr/lib"]

modificados = []
for ruta in rutas_criticas:
    salida = _ejecutar(
        ["find", ruta, "-xdev", "-mmin", "-1440",
         "-not", "-path", "*/proc/*",
         "-not", "-name", "*.log",
         "-type", "f"],
        timeout=15
    )
    for arch in salida.splitlines():
        arch = arch.strip()
        if arch:
            modificados.append(arch)

if modificados:
    sev = SEV_HIGH if len(modificados) > 5 else SEV_MEDIUM
    _print_finding(finding(sev, MOD,
        f"{len(modificados)} archivo(s) del sistema modificados en
últimas 24h",
        "\n".join(modificados)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin modificaciones recientes en
rutas del sistema"))

# World-writable en rutas críticas
ww = []
for ruta in ["/etc", "/bin", "/usr/bin", "/sbin"]:
    salida = _ejecutar(
        ["find", ruta, "-xdev", "-perm", "-o+w", "-type", "f"],
        timeout=10
    )
    for arch in salida.splitlines():
        arch = arch.strip()
        if arch:
            ww.append(arch)

if ww:
    _print_finding(finding(SEV_CRITICAL, MOD,
        f"{len(ww)} archivo(s) world-writable en rutas críticas",
        "\n".join(ww)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin archivos world-writable en
rutas críticas"))

```

```

# SUID/SGID inusuales
salida_suid = _ejecutar(
    ["find", "/", "-xdev", "-perm", "/6000", "-type", "f"],
    timeout=20
)
suid_malos = []
for ruta in salida_suid.splitlines():
    ruta = ruta.strip()
    if ruta and ruta not in SUID_ESPERADOS:
        suid_malos.append(ruta)
if suid_malos:
    _print_finding(finding(SEV_HIGH, MOD,
        f"{len(suid_malos)} binario(s) SUID/SGID inusual(es)",
        "\n".join(suid_malos)))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin binarios SUID/SGID
inusuales"))

# Archivos borrados abiertos por procesos (memoria forense básica)
salida_lsof = _ejecutar(["lsof", "+l1"], timeout=10)
borrados = [l for l in salida_lsof.splitlines() if "(deleted)" in l]
if borrados:
    _print_finding(finding(SEV_HIGH, MOD,
        f"{len(borrados)} archivo(s) borrado(s) aún abierto(s) por
procesos (técnica anti-forense)",
        "\n".join(borrados[:20])))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin archivos borrados abiertos
por procesos"))

# Preload inusual (LD_PRELOAD backdoor)
ld_preload = []
ld_so = _leer_archivo("/etc/ld.so.preload")
if ld_so and ld_so.strip():
    _print_finding(finding(SEV_CRITICAL, MOD,
        "/etc/ld.so.preload existe (posible rootkit/backdoor
LD_PRELOAD)",
        ld_so.strip()))
else:
    _print_finding(finding(SEV_OK, MOD, "Sin /etc/ld.so.preload"))

# Módulos del kernel cargados inusuales
salida_lsmod = _ejecutar(["lsmod"])
modulos = salida_lsmod.splitlines()[1:] if salida_lsmod else []
_print_finding(finding(SEV_INFO, MOD,

```

```

f"{len(modulos)} módulo(s) de kernel cargado(s)",
"\n".join(modulos[:30]) if modulos else "No disponible"))

# Historial bash de usuarios
for usuario, home, _ in _listar_usuarios_reales():
    hist_path = os.path.join(home, ".bash_history")
    contenido = _leer_archivo(hist_path)
    if not contenido:
        continue
    comandos_malos = []
    PATRONES_MALOS = [
        r"nc\s+-", r"ncat\s", r"curl.*\|\s*(ba)?sh",
r"wget.*\|\s*(ba)?sh",
        r"python.*-c.*socket", r"perl.*-e.*socket", r"base64\s+-d.*\|",
        r"chmod\s+[0-9]*7[0-9]*\s+/", r"/dev/tcp/", r"&\s*/dev/null",
    ]
    for cmd in contenido.splitlines():
        cmd = cmd.strip()
        if any(re.search(p, cmd) for p in PATRONES_MALOS):
            comandos_malos.append(f"[{usuario}] {cmd}")
    if comandos_malos:
        _print_finding(finding(SEV_HIGH, MOD,
            f"Comandos sospechosos en historial de {usuario}",
            "\n".join(comandos_malos)))

# -----
# MÓDULO 5: INTEGRIDAD DE ARCHIVOS
# -----
def mod_integridad():
    MOD = "Integridad"
    sep("INTEGRIDAD DE ARCHIVOS CRÍTICOS")

    # Hashes actuales
    hashes_actuales = {}
    for path in ARCHIVOS_CRITICOS:
        h = _hash_archivo(path)
        hashes_actuales[path] = h or "NO_EXISTE/SIN_PERMISO"
        estado = SEV_OK if h and h != "SIN_PERMISO" else SEV_INFO
        _print_finding(finding(estado, MOD, f"Hash: {path}",
            hashes_actuales[path]))

    # Verificar contra baseline si existe
    if not os.path.exists(BASELINE_FILE):
        _print_finding(finding(SEV_INFO, MOD,
            "No existe baseline – usa opción 17 para generar uno"))

```



```

        return

    try:
        with open(BASELINE_FILE) as f:
            baseline = json.load(f)
    except Exception as e:
        _print_finding(finding(SEV_INFO, MOD, f"Error leyendo baseline:
{e}"))
        return

    alertas = 0
    for path, data in baseline.items():
        h_guardado = data["hash"]
        h_actual = hashes_actuales.get(path) or _hash_archivo(path) or
"NO_EXISTE"
        if h_actual != h_guardado:
            _print_finding(finding(SEV_CRITICAL, MOD,
                f"CAMBIO DE INTEGRIDAD: {path}",
                f"Guardado: {h_guardado[:48]}\nActual: {h_actual[:48]}"))
            alertas += 1

    if alertas == 0:
        _print_finding(finding(SEV_OK, MOD,
            f"Todos los {len(baseline)} archivos del baseline sin cambios"))

def generar_baseline():
    sep("GENERANDO BASELINE")
    baseline = {}
    for path in ARCHIVOS_CRITICOS:
        h = _hash_archivo(path) or "NO_EXISTE"
        baseline[path] = {"hash": h, "timestamp":
datetime.datetime.now().isoformat()}
        estado = "OK" if h and h != "SIN_PERMISO" else "!"
        print(f" [{estado}] {path[:50]:50s} {h[:20]}...")
    with open(BASELINE_FILE, "w") as f:
        json.dump(baseline, f, indent=2)
    print(f"\n Baseline guardado: {os.path.abspath(BASELINE_FILE)}")

# -----
#   REPORTE HTML
# -----
HTML_TEMPLATE = """<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>SOC Report – {hostname} – {fecha}</title>
<style>
  :root {{
    --bg:      #0d1117;
    --bg2:     #161b22;
    --bg3:     #1c2128;
    --border:   #30363d;
    --text:     #e6edf3;
    --muted:    #8b949e;
    --crit:     #ff4444;
    --high:     #ff8c00;
    --med:      #e3b341;
    --info:     #58a6ff;
    --ok:       #3fb950;
    --font:     'JetBrains Mono', 'Fira Code', 'Courier New', monospace;
  }}
  * {{ box-sizing: border-box; margin: 0; padding: 0; }}
  body {{
    background: var(--bg);
    color: var(--text);
    font-family: var(--font);
    font-size: 13px;
    line-height: 1.6;
  }}
  header {{
    background: var(--bg2);
    border-bottom: 1px solid var(--border);
    padding: 24px 40px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    position: sticky;
    top: 0;
    z-index: 100;
  }}
  header h1 {{ font-size: 18px; letter-spacing: 0.05em; color: var(--text); }}
  header .meta {{ color: var(--muted); font-size: 11px; text-align: right; }}
  .layout {{ display: flex; min-height: 100vh; }}
  nav {{
    width: 220px;
    min-width: 220px;
    background: var(--bg2);

```

```

border-right: 1px solid var(--border);
padding: 20px 0;
position: sticky;
top: 65px;
height: calc(100vh - 65px);
overflow-y: auto;
}}
nav h3 {{
font-size: 10px;
letter-spacing: 0.1em;
color: var(--muted);
padding: 8px 20px 4px;
text-transform: uppercase;
}}
nav a {{
display: block;
padding: 6px 20px;
color: var(--muted);
text-decoration: none;
font-size: 12px;
border-left: 2px solid transparent;
transition: all 0.15s;
}}
nav a:hover, nav a.active {{
color: var(--text);
border-left-color: var(--info);
background: var(--bg3);
}}
main {{
flex: 1;
padding: 32px 40px;
max-width: 1100px;
}}

/* Summary cards */
.summary {{
display: grid;
grid-template-columns: repeat(5, 1fr);
gap: 12px;
margin-bottom: 36px;
}}
.card {{
background: var(--bg2);
border: 1px solid var(--border);
border-radius: 8px;

```

```

padding: 16px;
text-align: center;
}}
.card .num {{ font-size: 28px; font-weight: 700; }}
.card .label {{ font-size: 10px; color: var(--muted); margin-top: 4px;
text-transform: uppercase; letter-spacing: 0.08em; }}
.card.crit {{ border-top: 3px solid var(--crit); }} .card.crit .num {{
color: var(--crit); }}
.card.high {{ border-top: 3px solid var(--high); }} .card.high .num {{
color: var(--high); }}
.card.med {{ border-top: 3px solid var(--med); }} .card.med .num {{
color: var(--med); }}
.card.info {{ border-top: 3px solid var(--info); }} .card.info .num {{
color: var(--info); }}
.card.ok {{ border-top: 3px solid var(--ok); }} .card.ok .num {{
color: var(--ok); }}

/* Module sections */
.module {{ margin-bottom: 40px; }}
.module h2 {{
font-size: 14px;
letter-spacing: 0.05em;
text-transform: uppercase;
color: var(--text);
border-bottom: 1px solid var(--border);
padding-bottom: 10px;
margin-bottom: 16px;
}}

/* Findings */
.finding {{
background: var(--bg2);
border: 1px solid var(--border);
border-left: 3px solid var(--border);
border-radius: 4px;
margin-bottom: 8px;
overflow: hidden;
}}
.finding.CRÍTICO {{ border-left-color: var(--crit); }}
.finding.ALTO {{ border-left-color: var(--high); }}
.finding.MEDIO {{ border-left-color: var(--med); }}
.finding.INFO {{ border-left-color: var(--info); }}
.finding.OK {{ border-left-color: var(--ok); }}

.finding-header {{

```

```

display: flex;
align-items: center;
gap: 12px;
padding: 10px 14px;
cursor: pointer;
user-select: none;
}}
.finding-header:hover {{ background: var(--bg3); }}

.badge {{
font-size: 10px;
font-weight: 700;
letter-spacing: 0.08em;
padding: 2px 8px;
border-radius: 3px;
min-width: 70px;
text-align: center;
flex-shrink: 0;
}}
.badge.CRÍTICO {{ background: #3d0000; color: var(--crit); }}
.badge.ALTO {{ background: #2d1a00; color: var(--high); }}
.badge.MEDIO {{ background: #2d2000; color: var(--med); }}
.badge.INFO {{ background: #001a3d; color: var(--info); }}
.badge.OK {{ background: #00200a; color: var(--ok); }}

.finding-title {{ flex: 1; color: var(--text); }}
.toggle {{ color: var(--muted); font-size: 10px; }}

.finding-detail {{
display: none;
padding: 10px 14px;
border-top: 1px solid var(--border);
background: var(--bg3);
white-space: pre-wrap;
word-break: break-all;
color: var(--muted);
font-size: 12px;
max-height: 400px;
overflow-y: auto;
}}
.finding-detail.open {{ display: block; }}

footer {{
text-align: center;
padding: 32px;

```



```

document.querySelectorAll('.finding-header').forEach(h => {{
  h.addEventListener('click', () => {{
    const d = h.nextElementSibling;
    if (d && d.classList.contains('finding-detail')) {{
      d.classList.toggle('open');
      h.querySelector('.toggle').textContent =
d.classList.contains('open') ? '▲' : '▼';
    }}
  }});
  }});
  // Auto-expand críticos y altos
  document.querySelectorAll('.finding.CRÍTICO .finding-detail, .finding.ALTO
.finding-detail').forEach(d => {{
    d.classList.add('open');
    const t = d.previousElementSibling.querySelector('.toggle');
    if (t) t.textContent = '▲';
  }});
  // Nav active highlight on scroll
  const sections = document.querySelectorAll('.module');
  const navLinks = document.querySelectorAll('nav a');
  const observer = new IntersectionObserver(entries => {{
    entries.forEach(e => {{
      if (e.isIntersecting) {{
        navLinks.forEach(a => a.classList.remove('active'));
        const link = document.querySelector(`nav
a[href="#${{e.target.id}}"]`);
        if (link) link.classList.add('active');
      }}
    }});
  }}, {{ threshold: 0.2 }});
  sections.forEach(s => observer.observe(s));
</script>
</body>
</html>""""

def _generar_html(findings_por_modulo):
    def esc(s):
        return (s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("'", "&quot;"))

    modulos_html = ""
    nav_links = ""
    modulos_unicos = list(dict.fromkeys(f.module for f in FINDINGS))

    for modulo in modulos_unicos:

```

```

mod_id = re.sub(r"^[a-z0-9]", "-", modulo.lower())
fs      = [f for f in FINDINGS if f.module == modulo]
fs_sort = sorted(fs, key=lambda x: SEV_ORDER.get(x.severity, 99))

nav_links += f'<a href="#{mod_id}">{modulo}</a>\n'

findings_html = ""
for f in fs_sort:
    has_detail = bool(f.detail.strip())
    toggle     = '<span class="toggle">▼</span>' if has_detail else ""

    detail_blk = (f'<div class="finding-
detail">{esc(f.detail)}</div>'
                  if has_detail else "")
    findings_html += f"""
<div class="finding {f.severity}">
  <div class="finding-header">
    <span class="badge {f.severity}">{f.severity}</span>
    <span class="finding-title">{esc(f.title)}</span>
    {toggle}
  </div>
  {detail_blk}
</div>"""

    modules_html += f"""
<div class="module" id="{mod_id}">
  <h2>{esc(modulo)}</h2>
  {findings_html}
</div>"""

cnt = lambda s: sum(1 for f in FINDINGS if f.severity == s)

return HTML_TEMPLATE.format(
    version    = VERSION,
    hostname   = esc(os.uname().nodename),
    fecha      = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    euclid     = os.geteuid(),
    nav_links  = nav_links,
    modules_html = modules_html,
    cnt_crit   = cnt(SEV_CRITICAL),
    cnt_high   = cnt(SEV_HIGH),
    cnt_med    = cnt(SEV_MEDIUM),
    cnt_info   = cnt(SEV_INFO),
    cnt_ok     = cnt(SEV_OK),
)

```

```

# -----
#   REPORTE COMPLETO
# -----

TODOS_LOS_MODULOS = [
    ("SSH/Auth Logs",      mod_ssh_logs),
    ("Procesos y Red",     mod_procesos_red),
    ("Usuarios y Privilegios", mod_usuarios_privilegios),
    ("Forense",           mod_forense),
    ("Integridad",        mod_integridad),
]

def ejecutar_todos():
    for titulo, func in TODOS_LOS_MODULOS:
        print(f"\n → {titulo}...")
        try:
            func()
        except Exception as e:
            _print_finding(finding(SEV_INFO, titulo, f"Error en módulo:
{e}"))

def generar_reporte(output_html=True, output_json=False, output_txt=True):
    global FINDINGS
    FINDINGS = []

    ts      = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    host    = os.uname().nodename
    prefix = f"soc_{host}_{ts}"

    print(f"\n{col(C.BOLD, '*')} Iniciando análisis completo...\n")
    ejecutar_todos()

    archivos = []

    if output_txt:
        nombre_txt = f"{prefix}.txt"
        with open(nombre_txt, "w", encoding="utf-8") as r:
            r.write(f"SOC PUNTUAL v{VERSION} – REPORTE\n")
            r.write(f"Host: {host} | {datetime.datetime.now().isoformat()} |
euid={os.geteuid()}\n")
            r.write("=" * 60 + "\n\n")
            modulo_actual = None
            for f in FINDINGS:
                if f.module != modulo_actual:
                    modulo_actual = f.module

```

```

        r.write(f"\n{'-'*50}\n {f.module.upper()}\n{'-'*50}\n")
        r.write(f" [{f.severity:8s}] {f.title}\n")
        if f.detail:
            for l in f.detail.splitlines():
                r.write(f" {l}\n")
            r.write("\n" + "=" * 60 + "\n")
        archivos.append(nombre_txt)
        print(f" ✓ TXT: {os.path.abspath(nombre_txt)}")

if output_html:
    nombre_html = f"{prefix}.html"
    with open(nombre_html, "w", encoding="utf-8") as h:
        h.write(_generar_html(FINDINGS))
    archivos.append(nombre_html)
    print(f" ✓ HTML: {os.path.abspath(nombre_html)}")

if output_json:
    nombre_json = f"{prefix}.json"
    with open(nombre_json, "w", encoding="utf-8") as j:
        json.dump({
            "metadata": {
                "version": VERSION, "host": host,
                "timestamp": datetime.datetime.now().isoformat(),
                "euid": os.geteuid(),
            },
            "summary": {
                "CRÍTICO": sum(1 for f in FINDINGS if f.severity ==
SEV_CRITICAL),
                "ALTO": sum(1 for f in FINDINGS if f.severity ==
SEV_HIGH),
                "MEDIO": sum(1 for f in FINDINGS if f.severity ==
SEV_MEDIUM),
                "INFO": sum(1 for f in FINDINGS if f.severity ==
SEV_INFO),
                "OK": sum(1 for f in FINDINGS if f.severity ==
SEV_OK),
            },
            "findings": [
                {"severity": f.severity, "module": f.module,
                 "title": f.title, "detail": f.detail, "ts": f.ts}
                for f in FINDINGS
            ],
        }, j, indent=2, ensure_ascii=False)
    archivos.append(nombre_json)
    print(f" ✓ JSON: {os.path.abspath(nombre_json)}")

```



```

    parser.add_argument("--report", action="store_true", help="Modo no-
interactivo")
    parser.add_argument("--html", action="store_true", help="Generar
reporte HTML")
    parser.add_argument("--json", action="store_true", help="Generar
reporte JSON")
    parser.add_argument("--no-color", action="store_true", help="Sin colores
ANSI")
    args = parser.parse_args()

    if args.no_color:
        USE_COLOR = False

    if os.geteuid() != 0:
        print("\n[!] ADVERTENCIA: No se ejecuta como root.")
        print("    Algunos módulos tendrán acceso limitado.")
        print("    Recomendado: sudo python3 soc.py\n")

    if args.report:
        generar_reporte(output_html=args.html, output_json=args.json,
output_txt=True)
        return

    ACCIONES = {
        "1": lambda: (setattr(sys.modules[__name__], 'FINDINGS', []),
mod_ssh_logs()),
        "2": lambda: (setattr(sys.modules[__name__], 'FINDINGS', []),
mod_procesos_red()),
        "3": lambda: (setattr(sys.modules[__name__], 'FINDINGS', []),
mod_usuarios_privilegios()),
        "4": lambda: (setattr(sys.modules[__name__], 'FINDINGS', []),
mod_forense()),
        "5": lambda: (setattr(sys.modules[__name__], 'FINDINGS', []),
mod_integridad()),
        "6": generar_baseline,
        "7": lambda: generar_reporte(output_html=True, output_json=False),
        "8": lambda: generar_reporte(output_html=True, output_json=True),
    }

    print(BANNER)
    while True:
        print(MENU)
        try:
            op = input(f"    {col(C.BOLD, 'Opción')}: ").strip()
        except (KeyboardInterrupt, EOFError):

```

```

        print(f"\n{col(C.INFO, '['*')}] Saliendo.")
        break

    if op == "0":
        print(f"\n{col(C.INFO, '['*')}] Saliendo.")
        break
    elif op in ACCIONES:
        try:
            ACCIONES[op]()
        except Exception as e:
            print(f"\n  [!] Error inesperado: {e}")
    else:
        print("  [!] Opción inválida.")

    try:
        input(f"\n  {col(C.DIM, 'Presiona Enter para continuar...')}")
    except (KeyboardInterrupt, EOFError):
        break

if __name__ == "__main__":
    main()

```



El resultado de ejecutar el script de Python envía un reporte en .txt y otro en .html como lo muestran las imágenes a continuación.

SOC PUNTUAL v3.1 - Reporte de Seguridad Ubuntu Server 2024-04-10 10:38:09 uid=0

MÓDULOS

- SSH/Auth
- Procesos/Red
- Usuarios/Privilegios
- Forense
- Integridad

0 CRÍTICO

3 ALTO

0 MEDIO

9 INFO

35 OK

SSH/AUTH

- INFO 18 evento(s) de sudo
- OK Sin intentos SSH fallidos
- OK Sin IPs con fuerza bruta
- OK Sin accesos fuera de horario
- OK Sin creación de usuarios detectada
- OK Sin indicios de escaneo de red

PROCESOS/RED

- INFO 10 servicio(s) en escucha
- INFO 5 conexión(es) establecida(s)
- INFO Reglas iptables (114 entradas)
- INFO 141 tarea(s) programada(s) en cron
- OK Sin procesos sospechosos

SOC PUNTUAL v3.1 - Reporte de Seguridad Ubuntu Server 2024-04-10 10:38:09 uid=0

MÓDULOS

- SSH/Auth
- Procesos/Red
- Usuarios/Privilegios
- Forense
- Integridad

FORENSE

- ALTO 15 archivo(s) del sistema modificados en últimas 24h
- ALTO 12 binario(s) SUID/SGID inusual(es)
- ALTO Comandos sospechosos en historial de ozone
- INFO 104 módulo(s) de kernel cargado(s)
- OK Sin scripts/ejecutables en /tmp
- OK Sin directorios ocultos inusuales en homes
- OK Sin archivos world-writable en rutas críticas
- OK Sin archivos borrados abiertos por procesos
- OK Sin /etc/ld.so.preload

INTEGRIDAD

- OK Hash: /etc/passwd
- OK Hash: /etc/shadow
- OK Hash: /etc/group
- OK Hash: /etc/sudoers
- OK Hash: /etc/crontab

19. Conclusiones

La culminación de este proyecto demuestra de forma fehaciente que la automatización es un pilar indispensable en la ciberseguridad moderna. Mediante la implementación secuencial de los scripts desarrollados en Python, se ha logrado transformar un flujo de datos crudo y masivo en inteligencia de amenazas accionable, culminando en la centralización de un "mini SOC" automatizado capaz de auditar de forma autónoma el estado del servidor.

La revisión detallada de los laboratorios permite extraer tres conclusiones fundamentales:

- **Comprensión sobre abstracción (El factor Wazuh):** Si bien plataformas empresariales y de código abierto como Wazuh integran estas funciones de manera nativa y a gran escala, la creación de scripts propios desmitifica el funcionamiento interno de sus motores de reglas. Al haber programado la lógica de detección, el analista adquiere el criterio técnico necesario para entender, modificar y optimizar los decodificadores y las alertas de cualquier SIEM del mercado, dejando de ser un mero usuario de software para convertirse en un arquitecto de detecciones.
- **Optimización y fidelidad:** La implementación de filtros y "coladores inteligentes" dentro del código demostró ser la estrategia más efectiva para mitigar el ruido operativo y la fatiga de alertas. Esto permitió discriminar con precisión las actividades administrativas legítimas de los verdaderos vectores de ataque (como la escalada de privilegios o la inyección de puertas traseras).
- **Escalabilidad y adaptabilidad:** El uso de herramientas estándar del sistema operativo (como *iptables*, *cron* y el análisis de *auth.log*) evidencia que es posible desplegar una postura de seguridad robusta y de alta fidelidad sin la estricta necesidad de software privativo o de infraestructuras sobredimensionadas que comprometan el rendimiento del servidor.